

Sorting

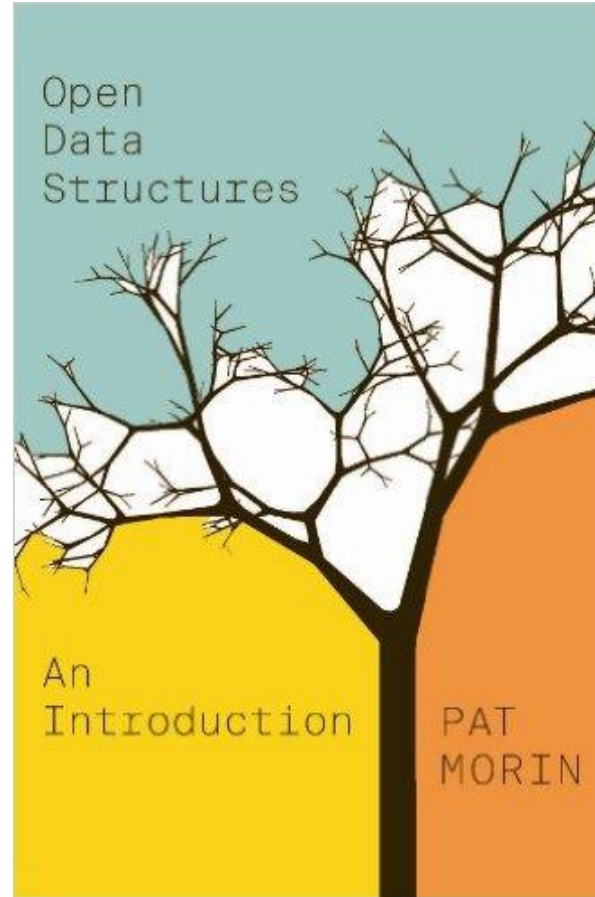
COMP 2402

Abstract Data Types & Algorithms

Readings

Today's class

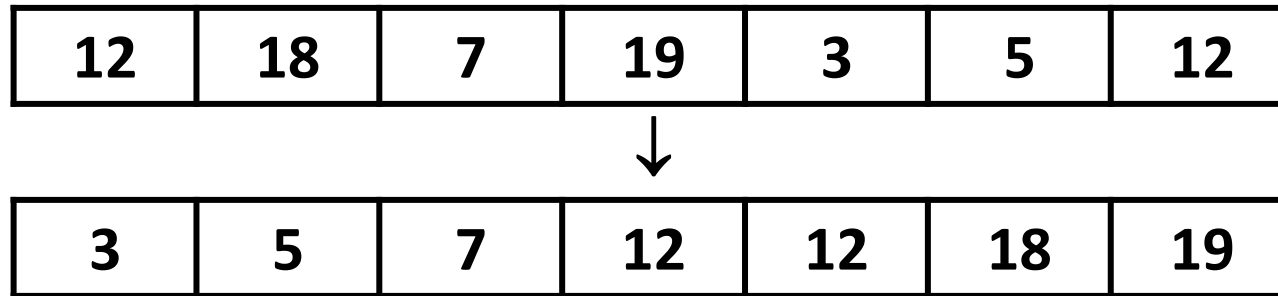
- Comparison-Based Sorting
 - Chapter 11.1
- Counting and Radix Sort
 - Chapter 11.2



Sorting

Comparison-Based Sorting

We'll look at several algorithms that order an array of items in non-decreasing order.



The (re)arrangement of the items is accomplished by repeatedly **comparing two items** in the array.

Sorting

Comparison-Based Sorting

The data we are sorting must come from a total order.

For any two elements x, y

$$\begin{matrix} x.\text{compareTo}(y) \\ \text{compare}(x, y) \end{matrix} = \begin{cases} < 0 & \text{when } x < y \\ 0 & \text{when } x = y \\ > 0 & \text{when } x > y \end{cases}$$

Mergesort

Theorem 11.1: Mergesort runs in $O(n \log n)$ time and performs at most $n \log n$ comparisons.

This is one of the classic example of a recursive divide & conquer algorithm.

Two observations needed for mergesort

1. A list with one element is already sorted
2. We can merge two sorted lists, of combined length n , to create a sorted list using $n - 1$ comparisons

Mergesort

Merge(A,B)

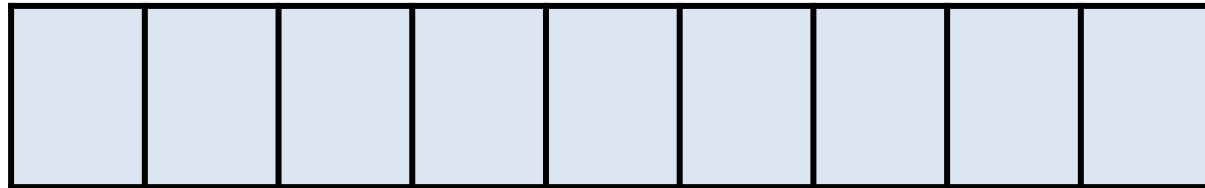
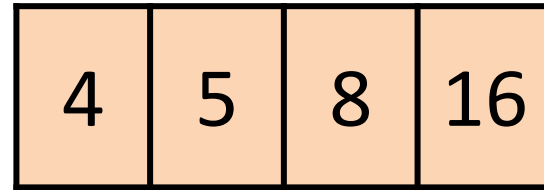
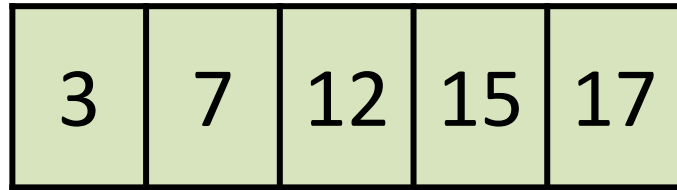
3	7	12	15	17
---	---	----	----	----

4	5	8	16
---	---	---	----

--	--	--	--	--	--	--	--	--

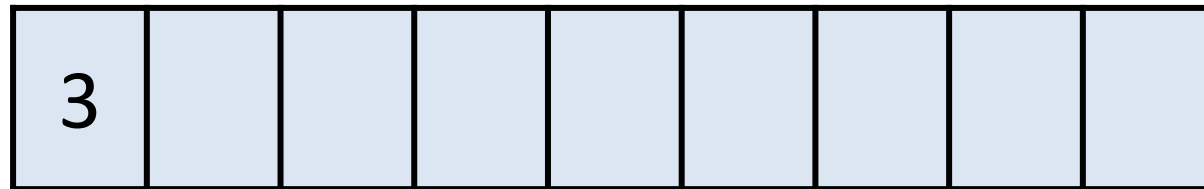
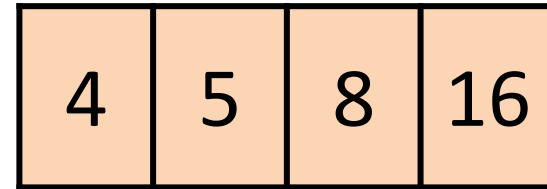
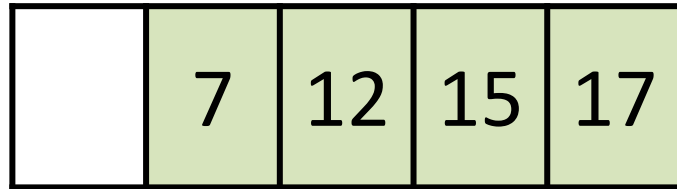
Mergesort

Merge(A,B)



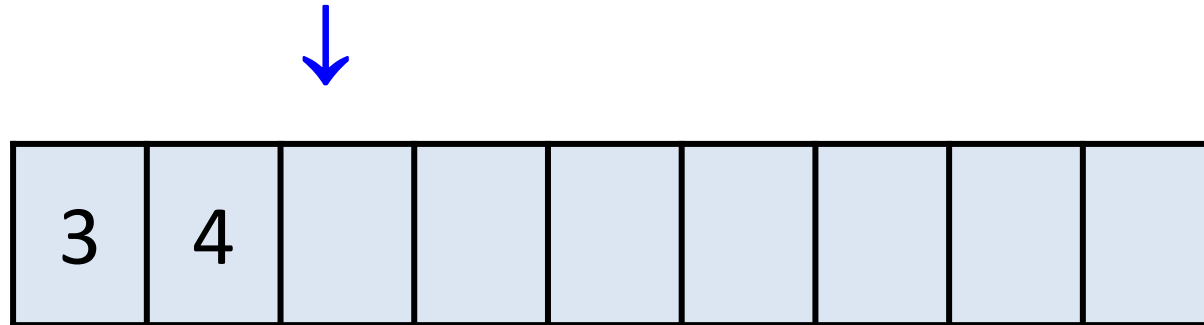
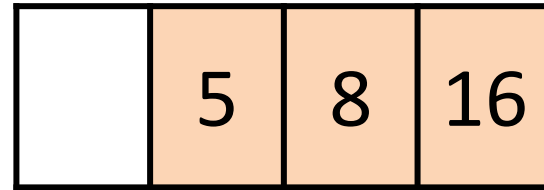
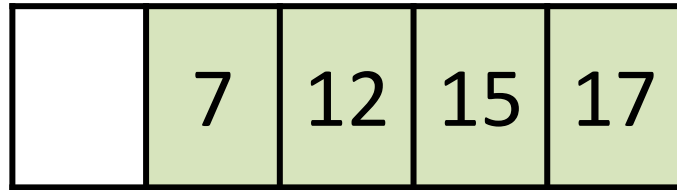
Mergesort

Merge(A,B)



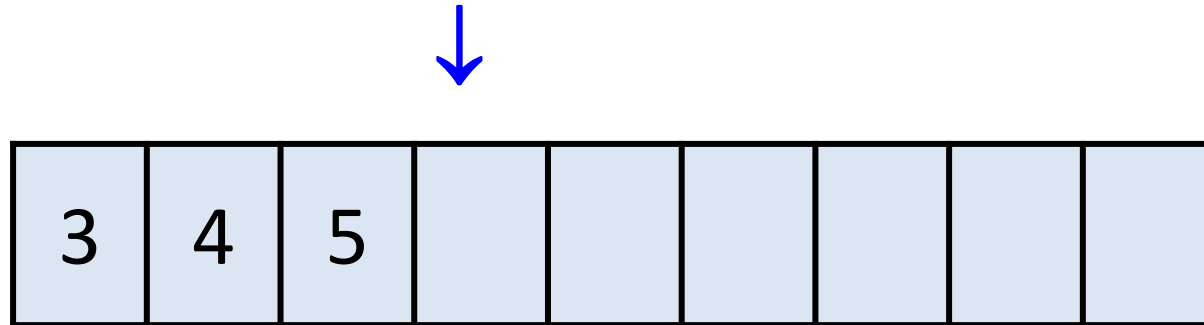
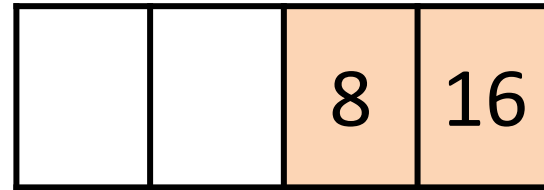
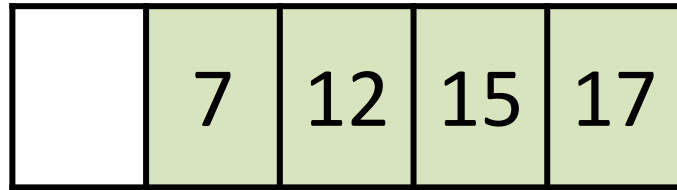
Mergesort

Merge(A,B)



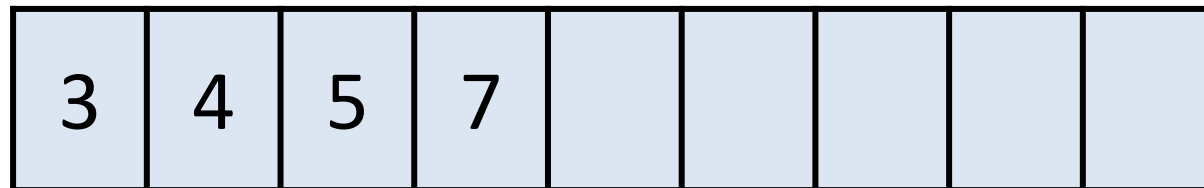
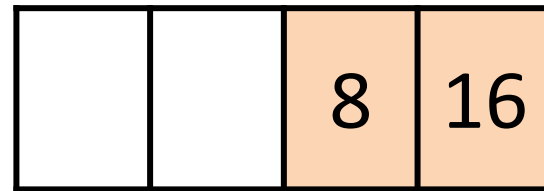
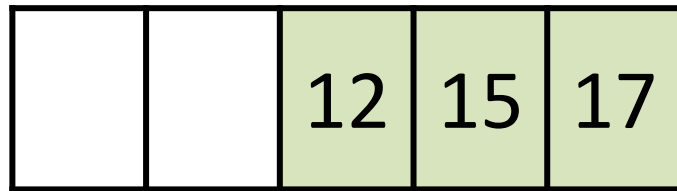
Mergesort

Merge(A,B)



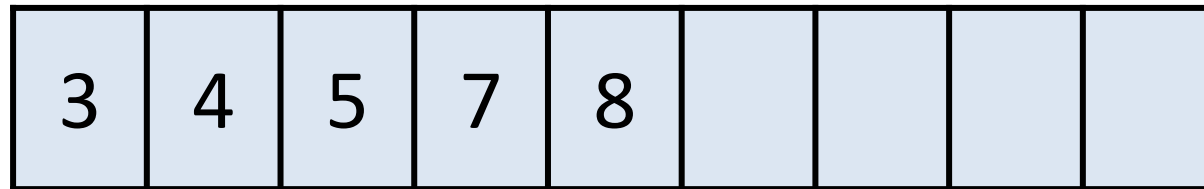
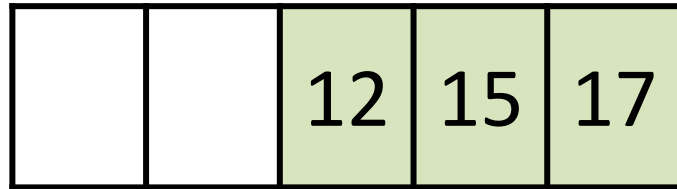
Mergesort

Merge(A,B)



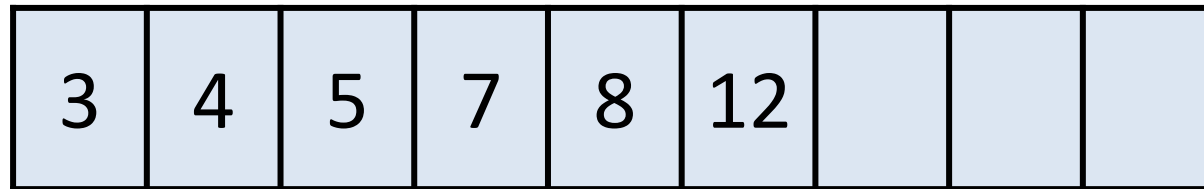
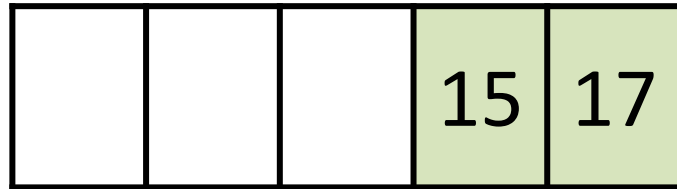
Mergesort

Merge(A,B)



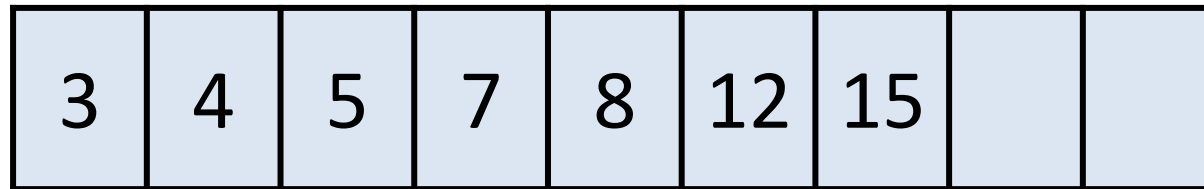
Mergesort

Merge(A,B)



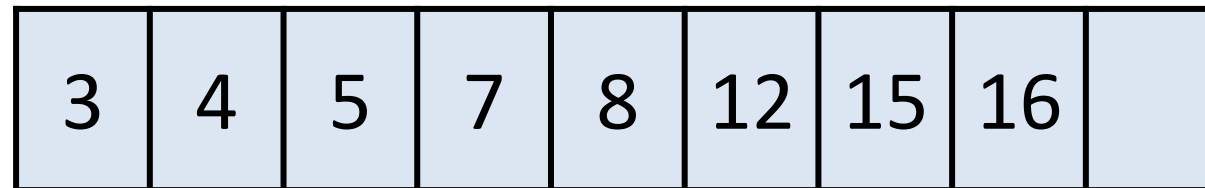
Mergesort

Merge(A,B)



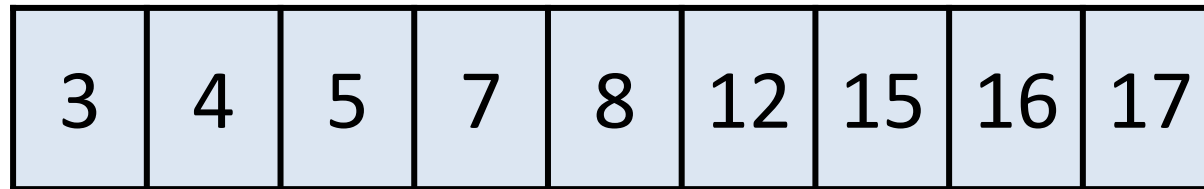
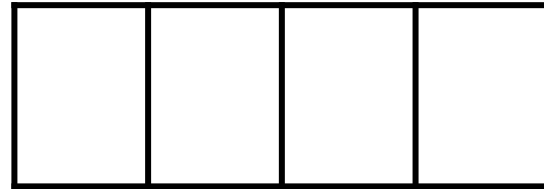
Mergesort

Merge(A,B)



Mergesort

Merge(A,B)



Mergesort

Merge(A,B)

3	4	5	7	8	12	15	16	17
---	---	---	---	---	----	----	----	----

Suppose $|A| = a$ and $|B| = b$ with $n = a + b$.

How many comparisons are needed in a call to merge(A,B)? What is the runtime?

Mergesort

Merge(A,B)

3	4	5	7	8	12	15	16	17
---	---	---	---	---	----	----	----	----

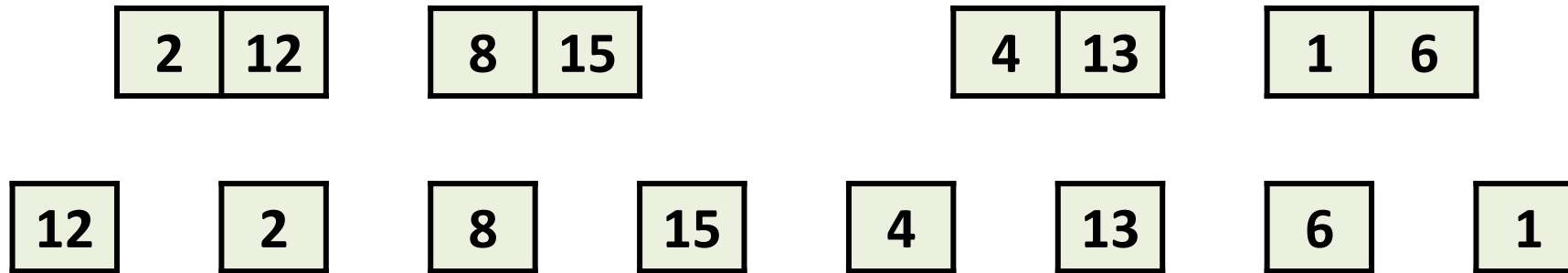
Suppose $|A| = a$ and $|B| = b$ with $n = a + b$.

merge(A,B) needs at most $n - 1$ comparisons
and runs in $O(n)$ time

Mergesort



Mergesort



Mergesort

1	8	12	15
---	---	----	----

1	4	6	14
---	---	---	----

2	12
---	----

8	15
---	----

4	13
---	----

1	6
---	---

12

2

8

15

4

13

6

1

Mergesort

1	2	4	6	8	12	14	15
---	---	---	---	---	----	----	----

2	8	12	15
---	---	----	----

1	4	6	14
---	---	---	----

2	12
---	----

8	15
---	----

4	13
---	----

1	6
---	---

12

2

8

15

4

13

6

1

Mergesort

mergesort(A)

// base case is singleton list

if A.size = 1 then return A

// recursively sort left and right sides of A

left := mergesort(left side of A)

right := mergesort(right side of A)

// combine these sorted lists into one

return merge(left, right)

Mergesort

```
mergesort(A[1..n])
```

```
  // base case is singleton list
```

```
  if A.size = 1 then return A
```

```
  // sort left and right sides of A
```

```
  left := mergesort( A[1..floor(n/2)])
```

```
  right := mergesort( A[floor(n/2)+1..n])
```

```
  return merge(left, right)
```


Mergesort

```
mergesort(A[0..n-1])
```

```
    // base case is singleton list
```

```
    if A.size = 1 then return A
```

```
    // sort left and right sides of A
```

```
    left := mergesort( A[0..floor(n/2)-1])
```

```
    right := mergesort( A[floor(n/2)..n-1])
```

```
    return merge(left, right)
```

Mergesort

mergesort(A)

12	4	7	15	3	13	3	14	1
----	---	---	----	---	----	---	----	---

Mergesort

mergesort(A)

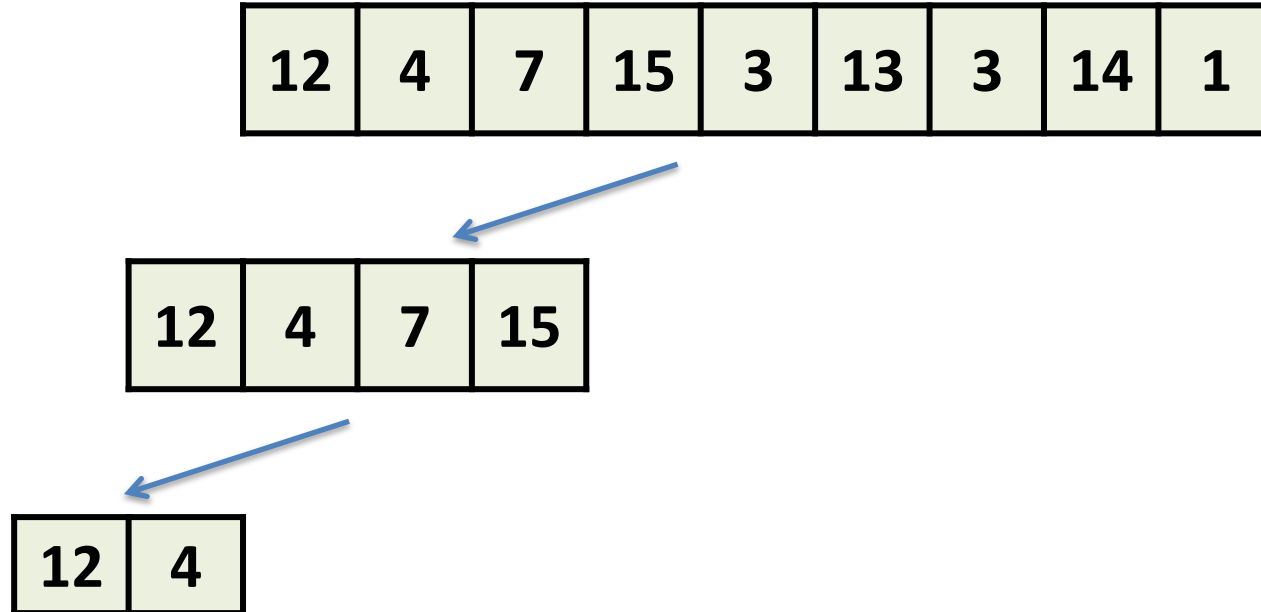
12	4	7	15	3	13	3	14	1
----	---	---	----	---	----	---	----	---



12	4	7	15
----	---	---	----

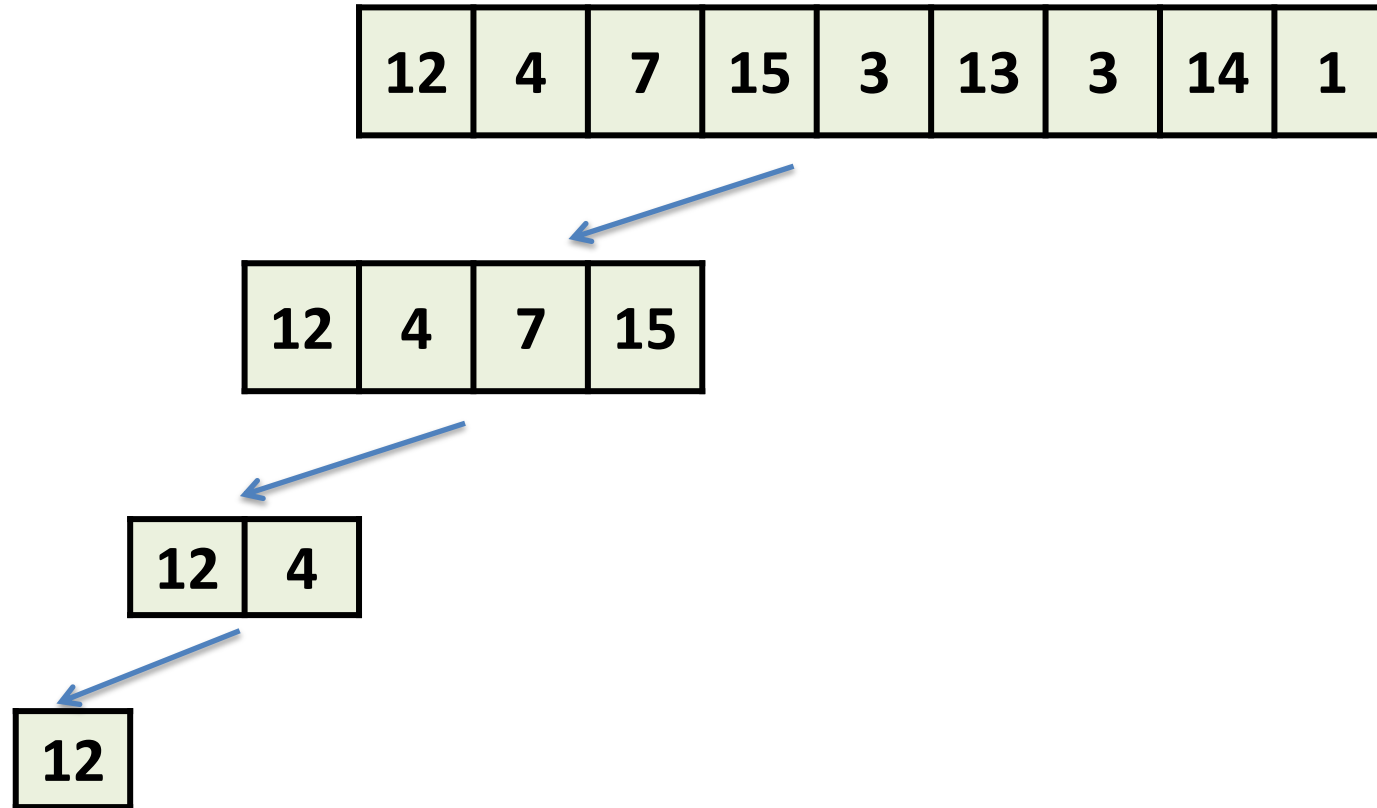
Mergesort

mergesort(A)



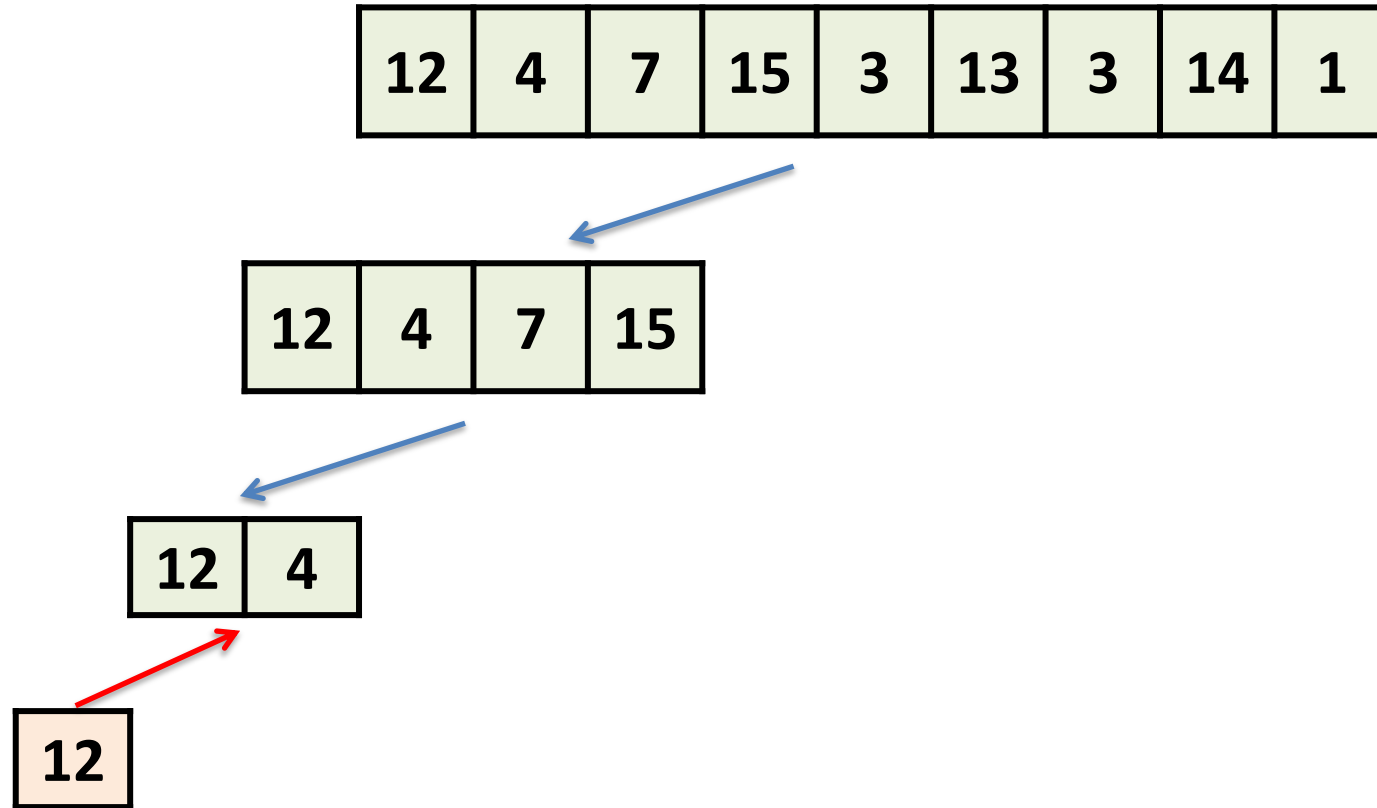
Mergesort

mergesort(A)



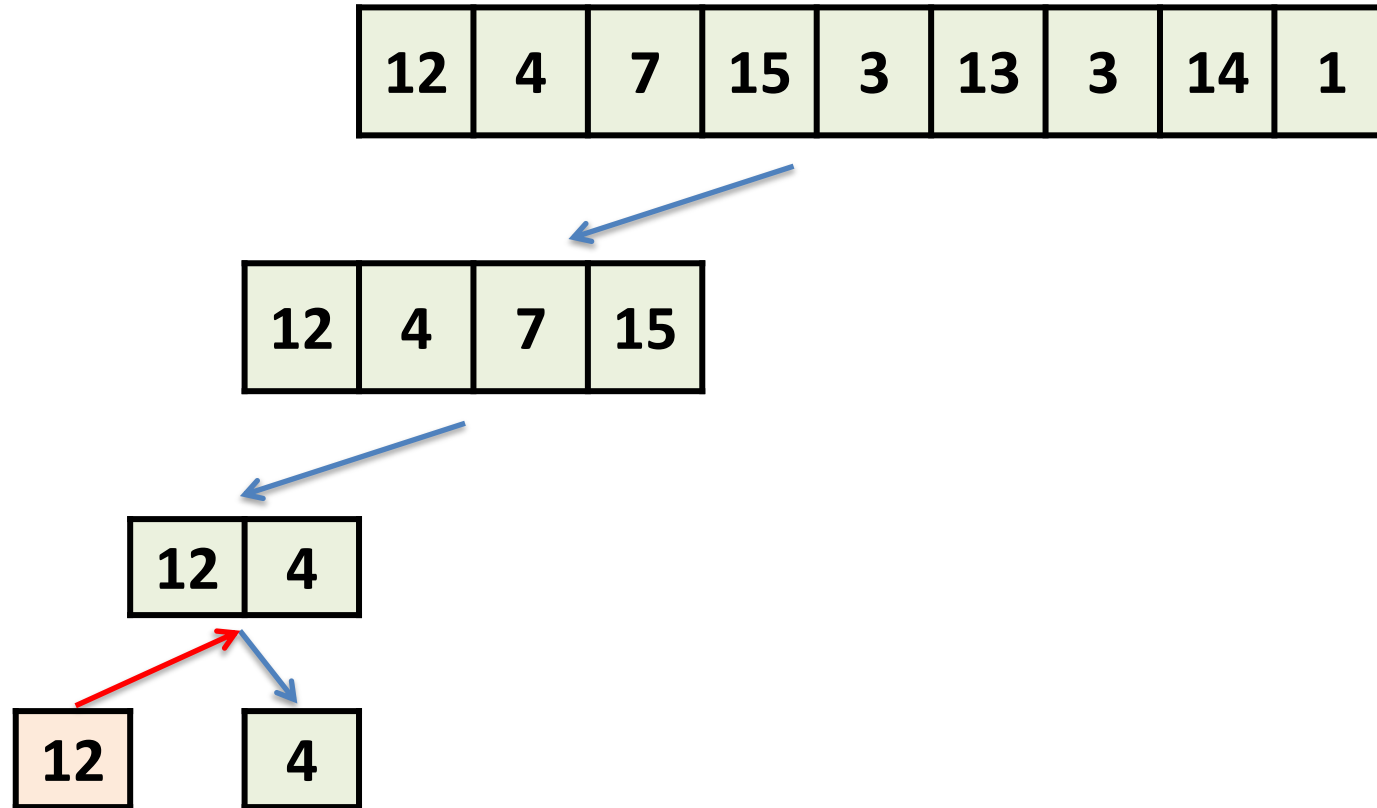
Mergesort

mergesort(A)



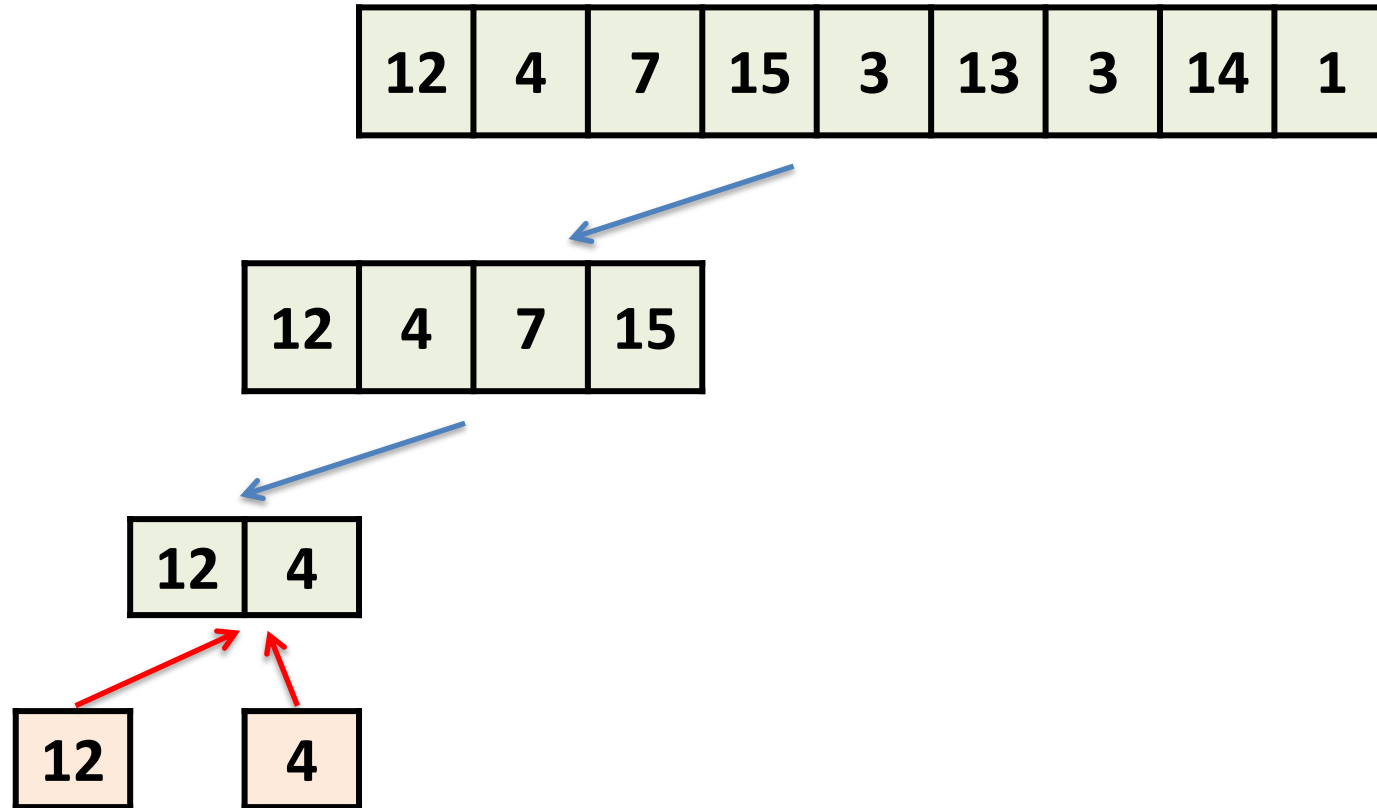
Mergesort

mergesort(A)



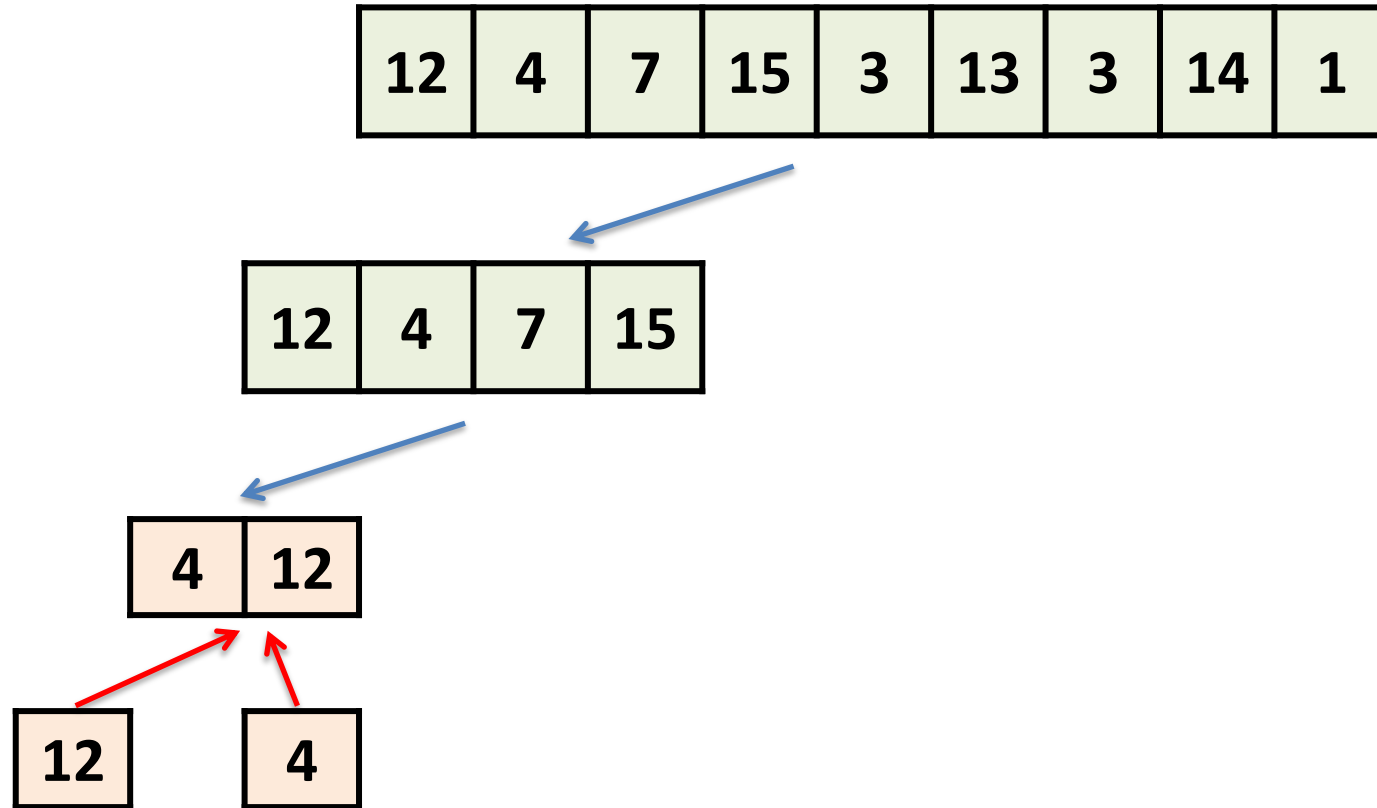
Mergesort

mergesort(A)



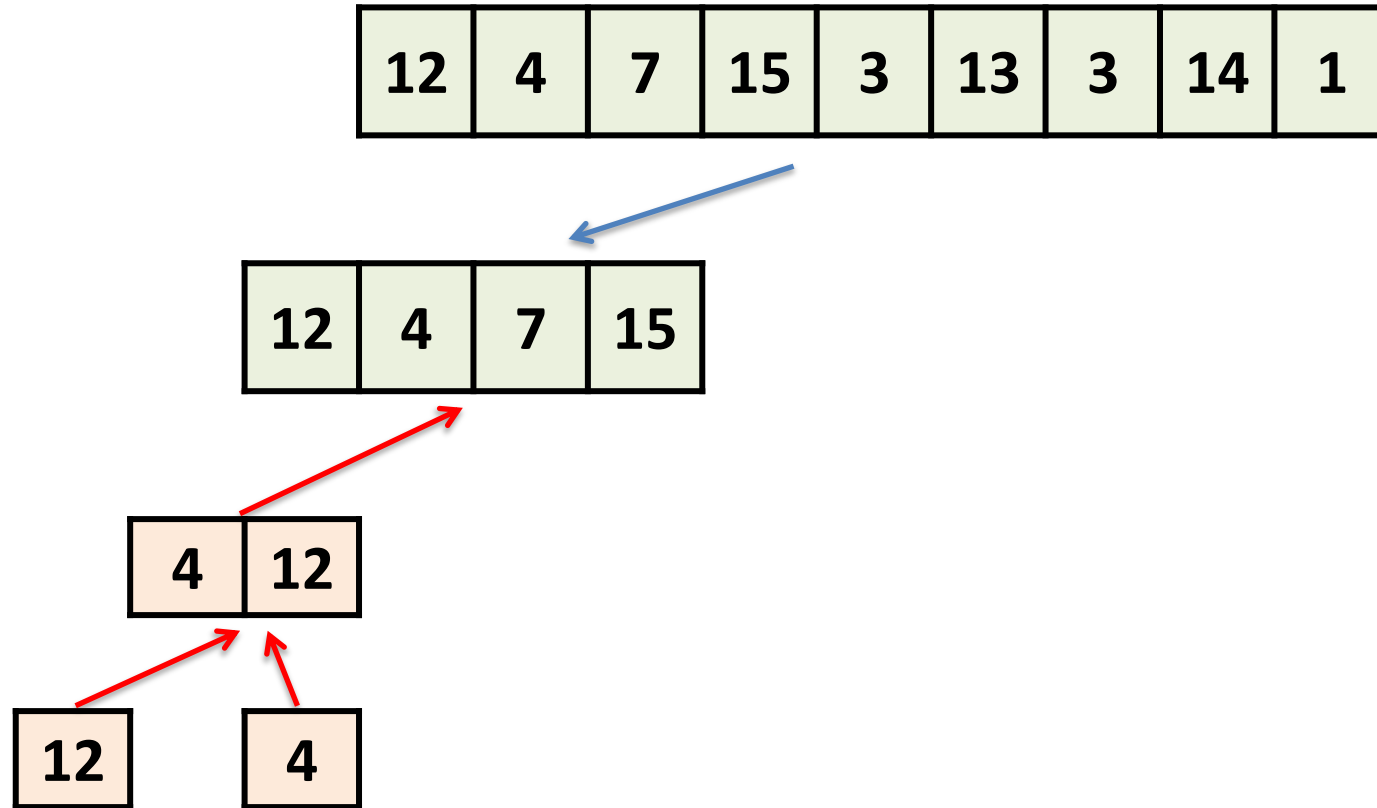
Mergesort

mergesort(A)



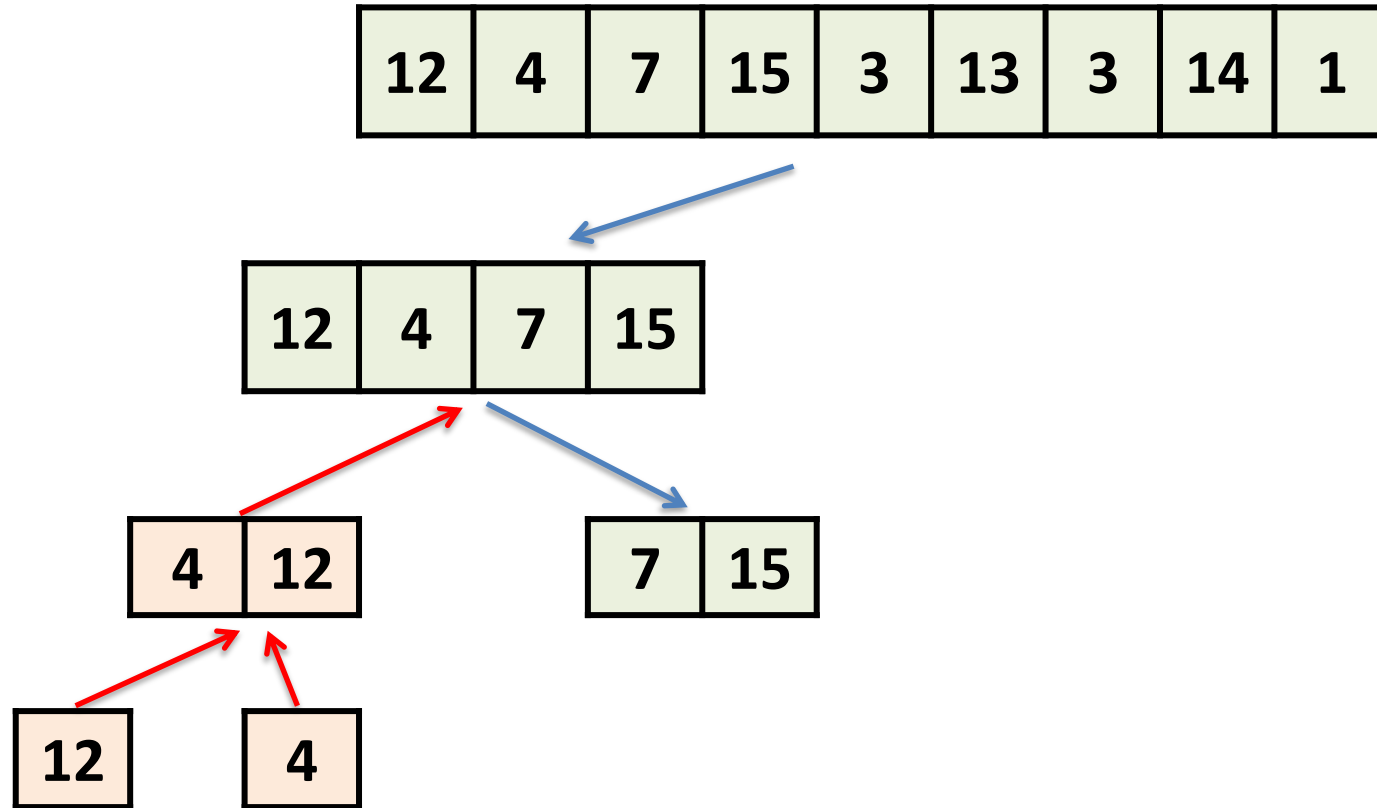
Mergesort

mergesort(A)



Mergesort

mergesort(A)



Mergesort

Let $C(n)$ be the number of comparisons needed to sort n elements with mergesort.

When $n = 1$:

$$C(1) = 0$$

When $n > 1$:

$$C(n) \leq (n - 1) + C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil)$$

Mergesort

Let $n = 2^m$

$$C(n) \leq (n - 1) + 2C(n/2)$$

Mergesort

Theorem 11.1: Mergesort runs in $O(n \log n)$ time and performs at most $n \log n$ comparisons.

The algorithm (as presented) needs $O(n \log n)$ extra space. We can do it with $n/2$ extra space.

Mergesort

Theorem 11.1: Mergesort runs in $O(n \log n)$ time and performs at most $n \log n$ comparisons.

The algorithm (as presented) needs $O(n \log n)$ extra space. We can do it with $n/2$ extra space.



this is NOT an in-place sorting algorithm

Mergesort

An **in-place** sorting algorithm uses $O(1)$ extra space. It does the sorting directly inside the array.

A **stable** sorting algorithm preserves the relative ordering of duplicates in the array. That is, duplicates appear in the same order before and after sorting.

For mergesort, always choosing the left duplicate first when merging results in a stable algorithm.

Quicksort

Another classic example of a recursive divide & conquer algorithm.

The main observations needed for quicksort

1. We can *partition* a list in linear time

`partition(A, p)` rearranges *A* so that all elements less than the pivot *p* appear before *p* and all elements greater than the pivot *p* appear after *p*.

Quicksort

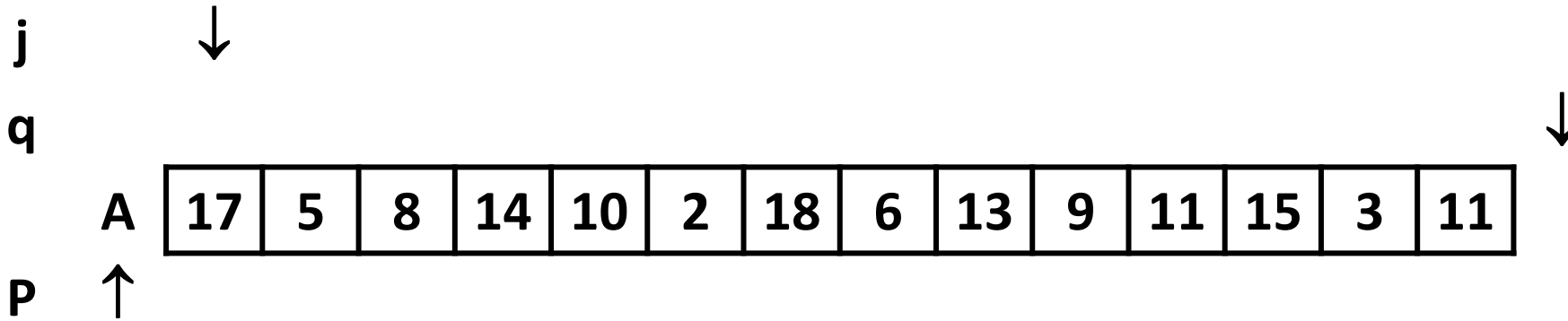
Theorem 11.3: Quicksort runs in $O(n \log n)$ **expected** time and the expected number of comparisons is at most $2n \ln(n) + O(n)$.

It has $O(n^2)$ worst-case runtime. It is an in-place sorting algorithm that is *not* stable. In practice, quicksort is very fast.

Note: We can modify the algorithm to guarantee $O(n \log(n))$ runtime in the worst-case. [COMP3804]

Quicksort

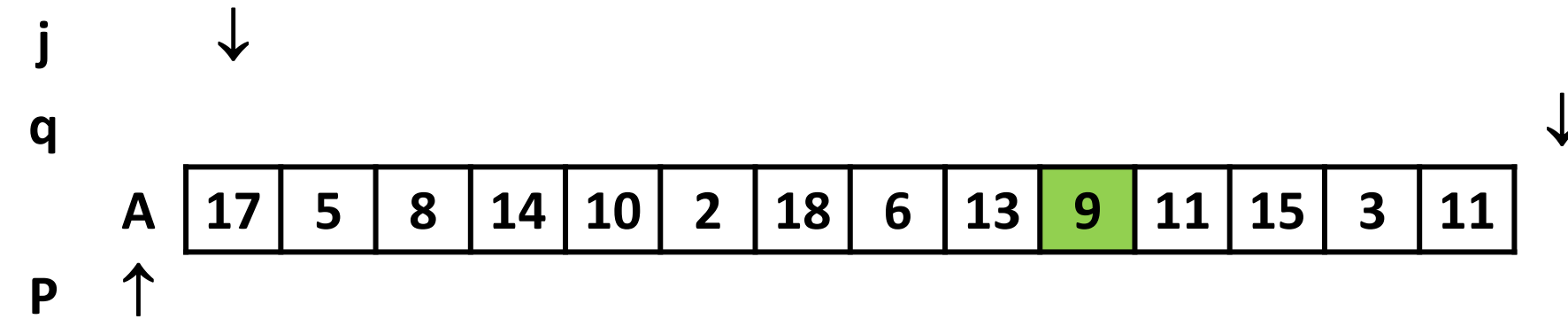
An efficient in-place partition function



Quicksort

An efficient in-place partition function

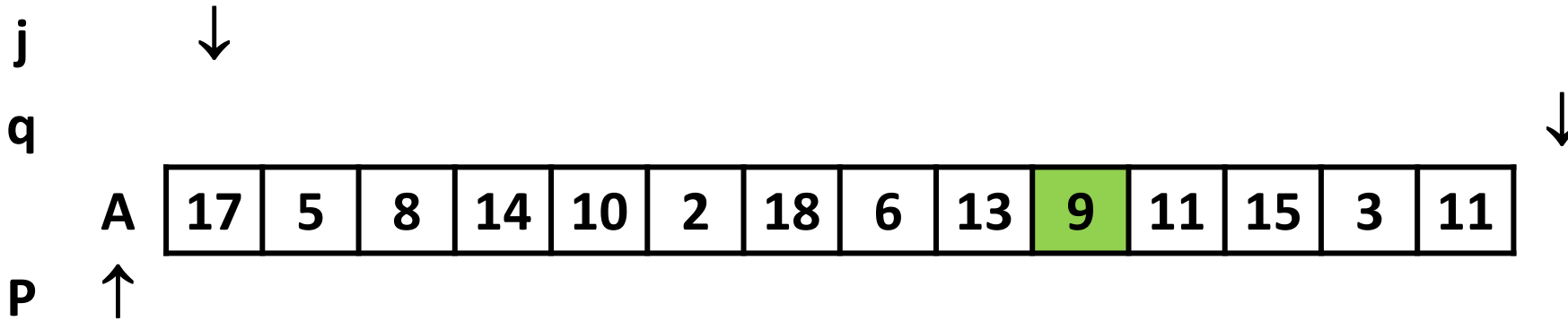
Pivot is $x = 9$



Quicksort

An efficient in-place partition function

Pivot is $x = 9$

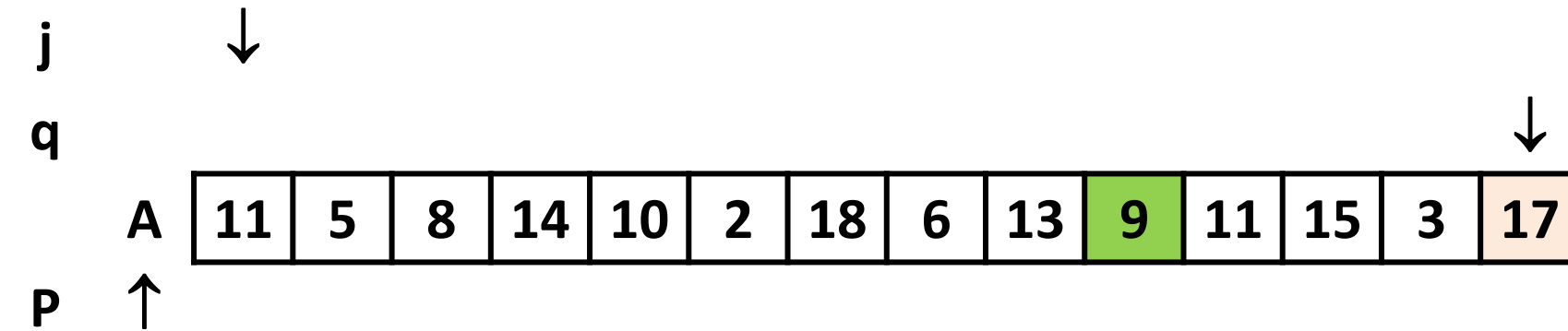


$A[j] = 17 > 9$ so swap $A[j]$ with $A[--q]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

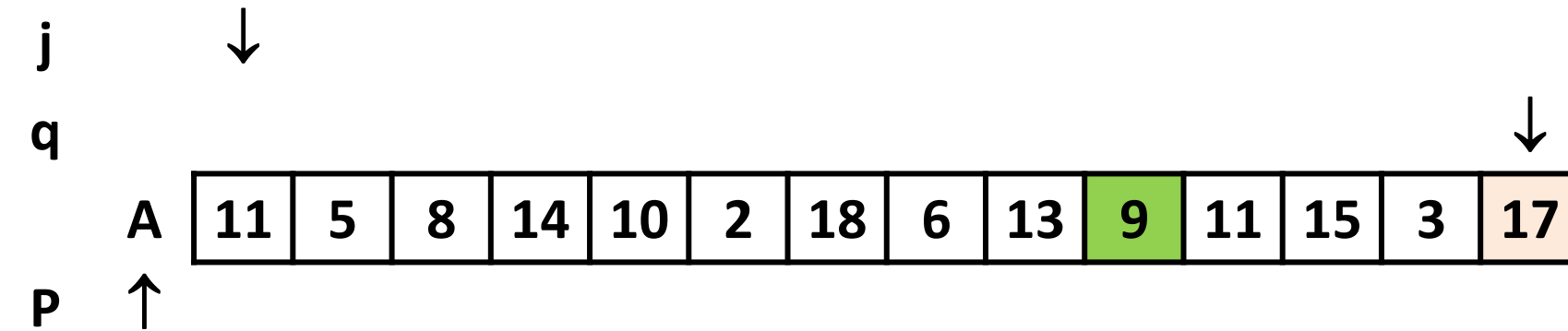


$A[j] = 17 > 9$ so swap $A[j]$ with $A[--q]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

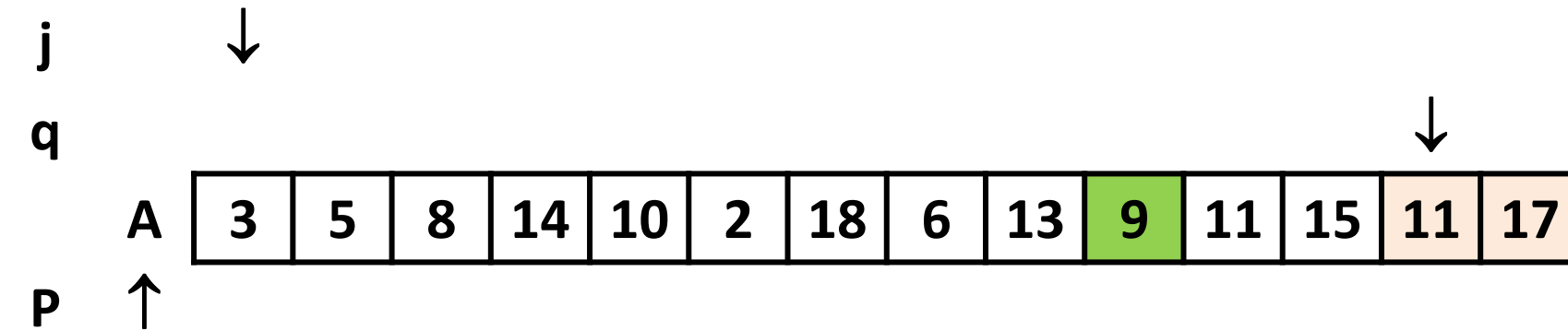


$A[j] = 11 > 9$ so swap $A[j]$ with $A[--q]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

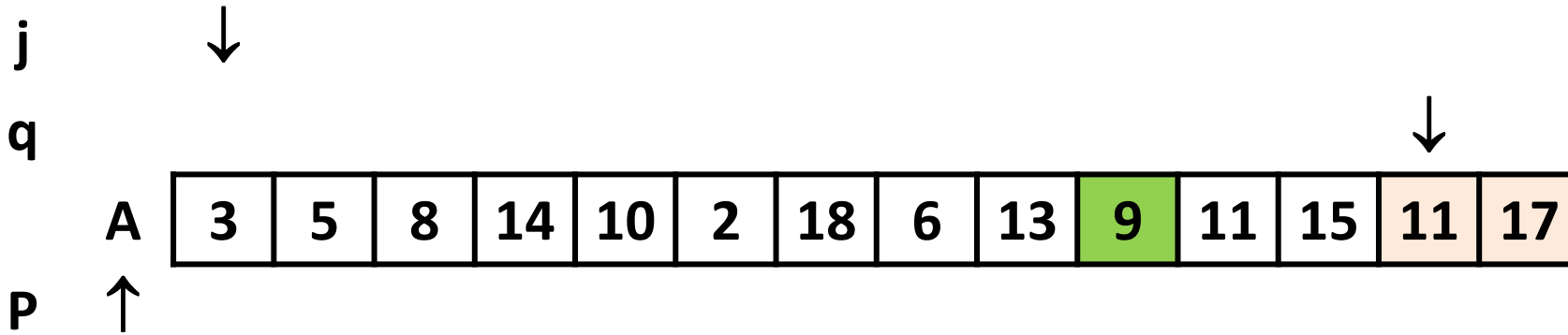


$A[j] = 11 > 9$ so swap $A[j]$ with $A[--q]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

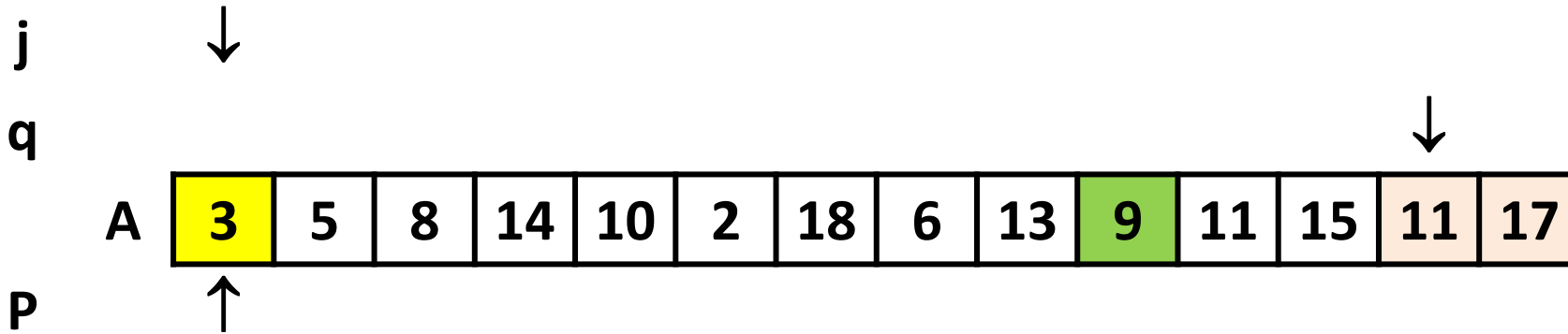


$A[j] = 3 < 9$ so swap $A[j++]$ with $A[++p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

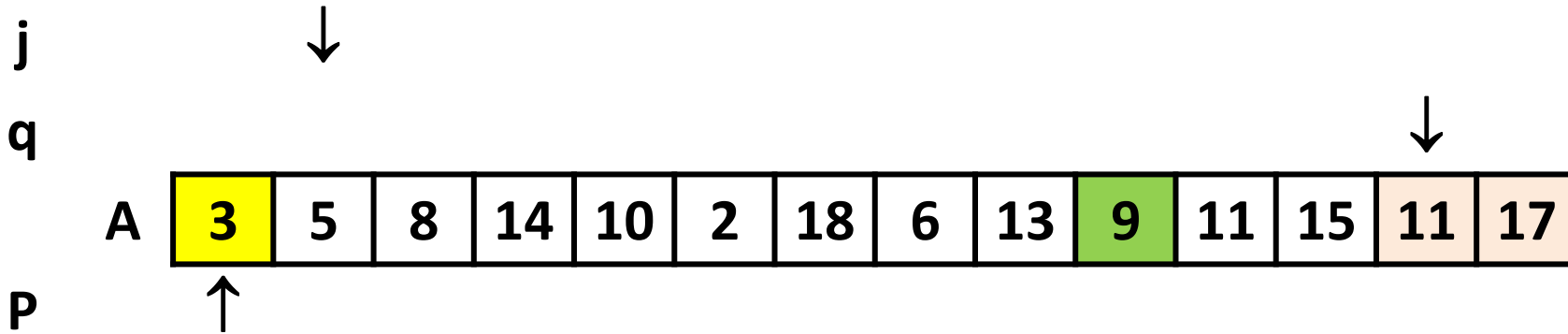


$A[j] = 3 < 9$ so swap $A[j++]$ with $A[++p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

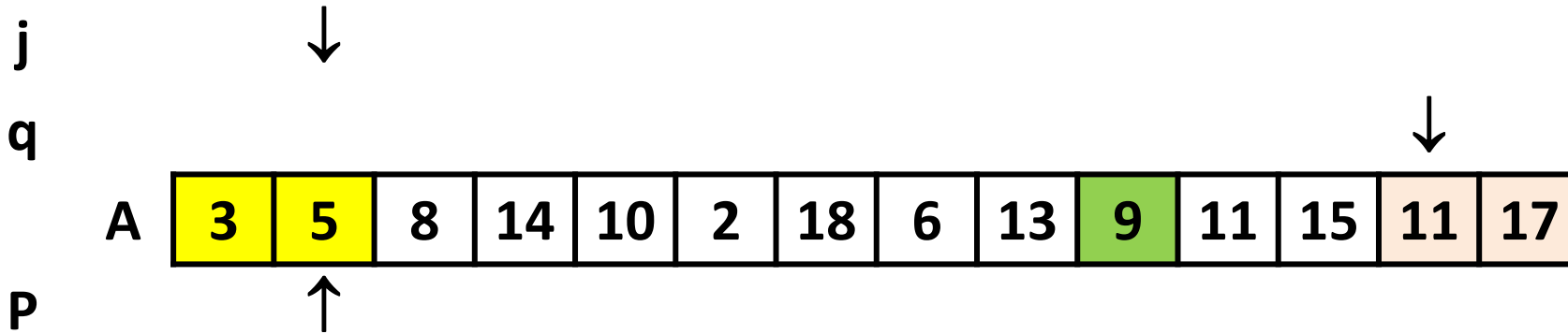


$A[j] = 5 < 9$ so swap $A[j++]$ with $A[++p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

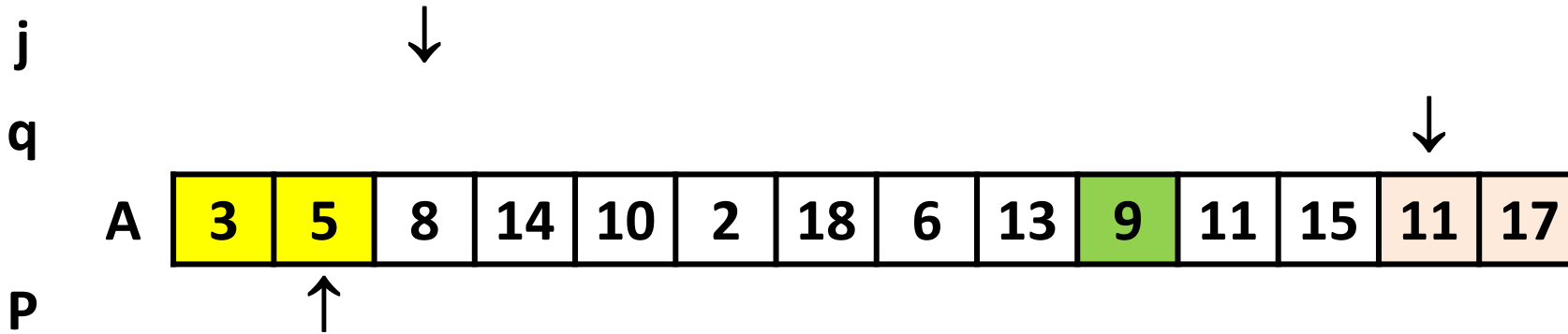


$A[j] = 5 < 9$ so swap $A[j++]$ with $A[++p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

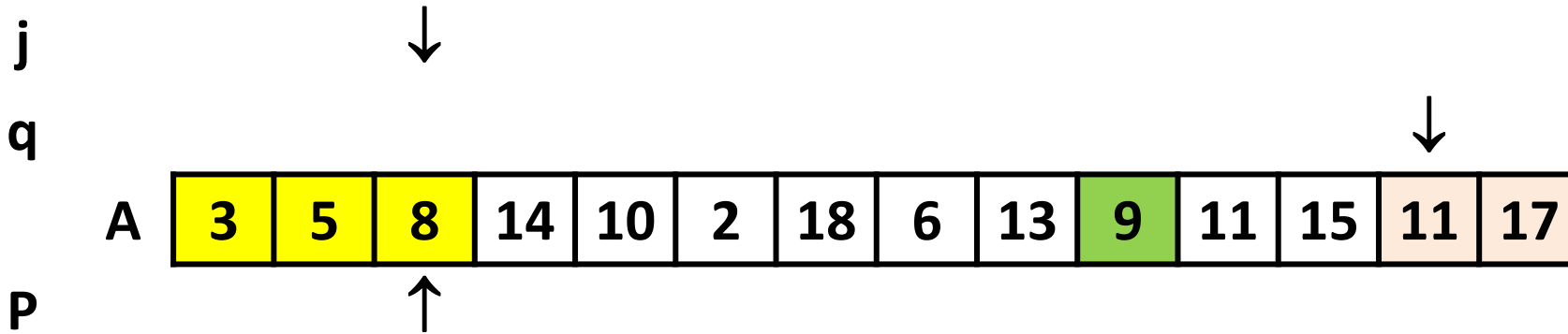


$A[j] = 8 < 9$ so swap $A[j++]$ with $A[++p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

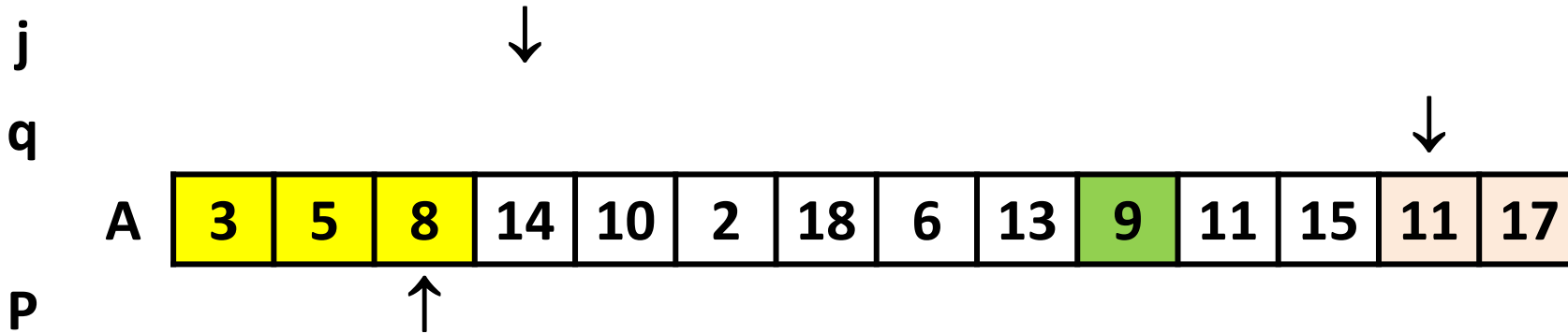


$A[j] = 8 < 9$ so swap $A[j++]$ with $A[++p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

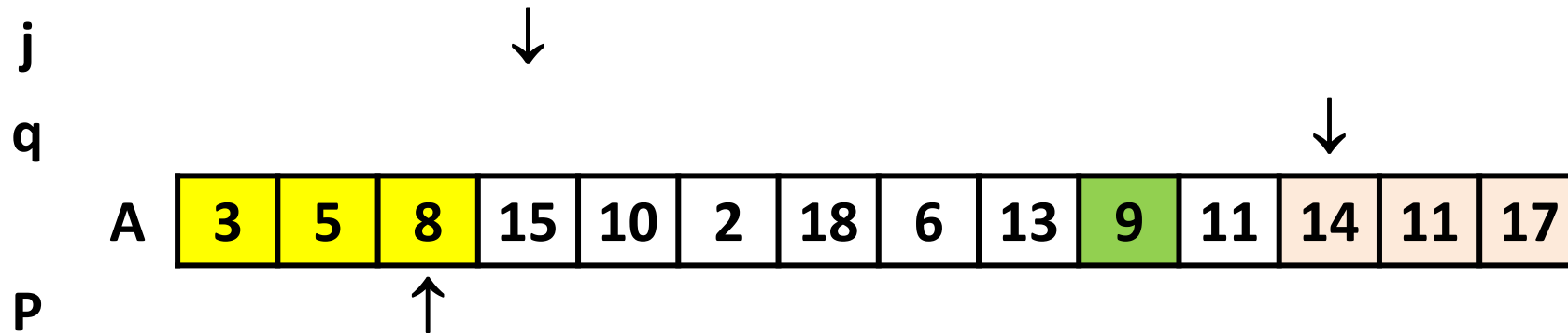


$A[j] = 14 > 9$ so swap $A[j]$ with $A[--p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

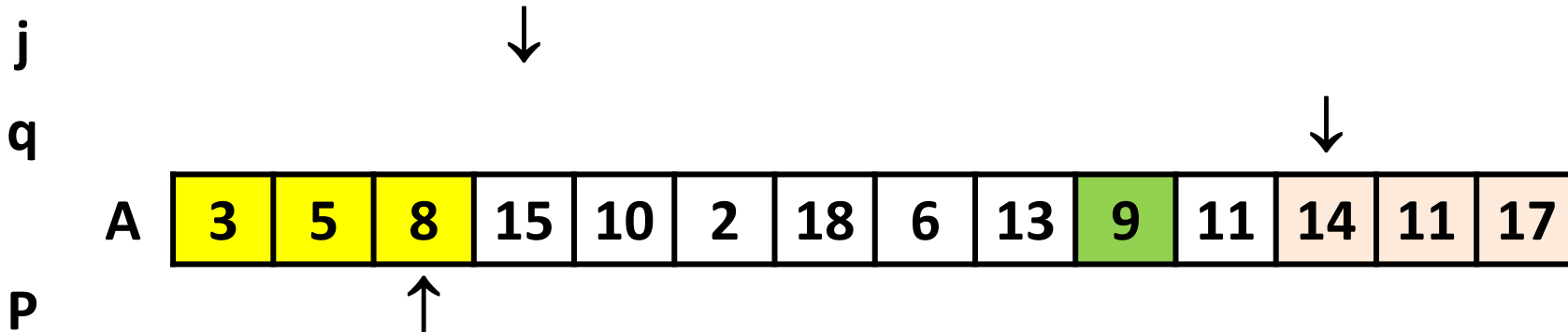


$A[j] = 14 > 9$ so swap $A[j]$ with $A[--p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

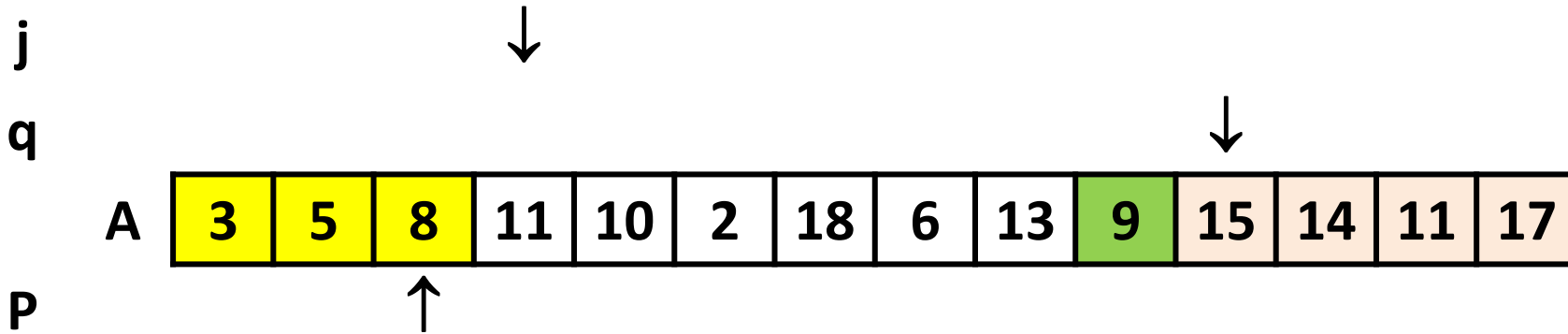


$A[j] = 15 > 9$ so swap $A[j]$ with $A[--p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

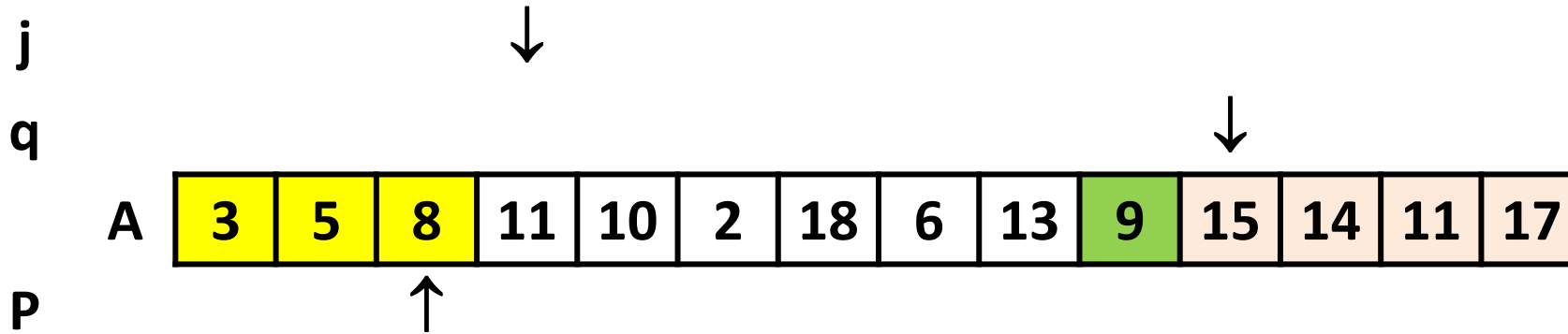


$A[j] = 15 > 9$ so swap $A[j]$ with $A[--p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

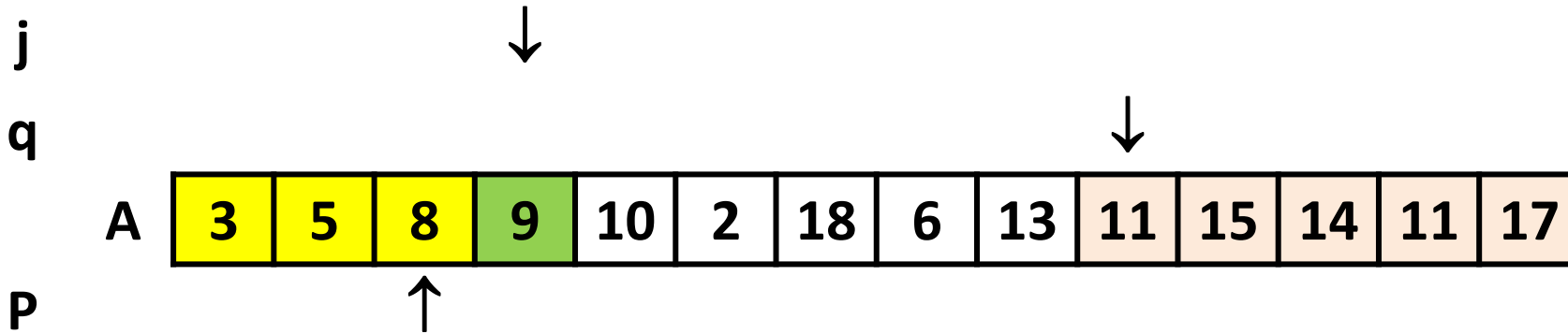


$A[j] = 11 > 9$ so swap $A[j]$ with $A[--p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

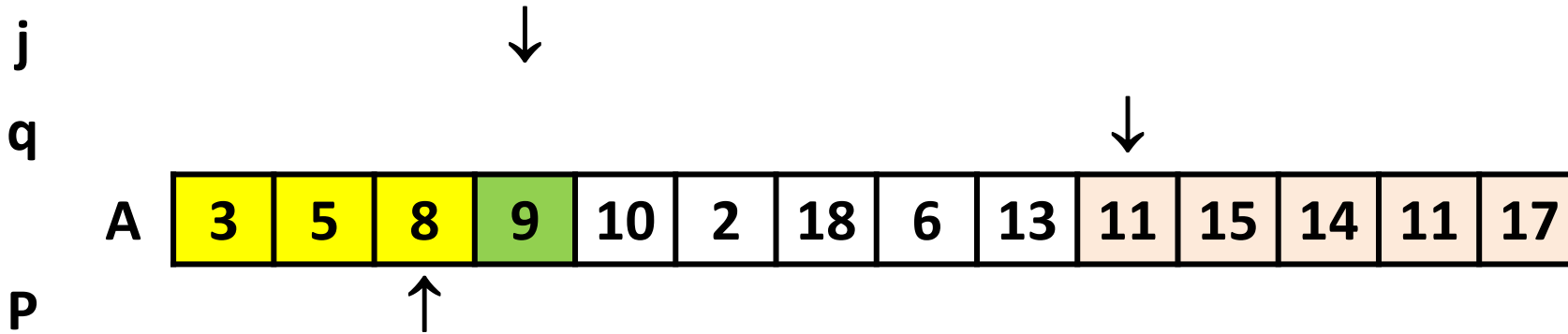


$A[j] = 11 > 9$ so swap $A[j]$ with $A[--p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

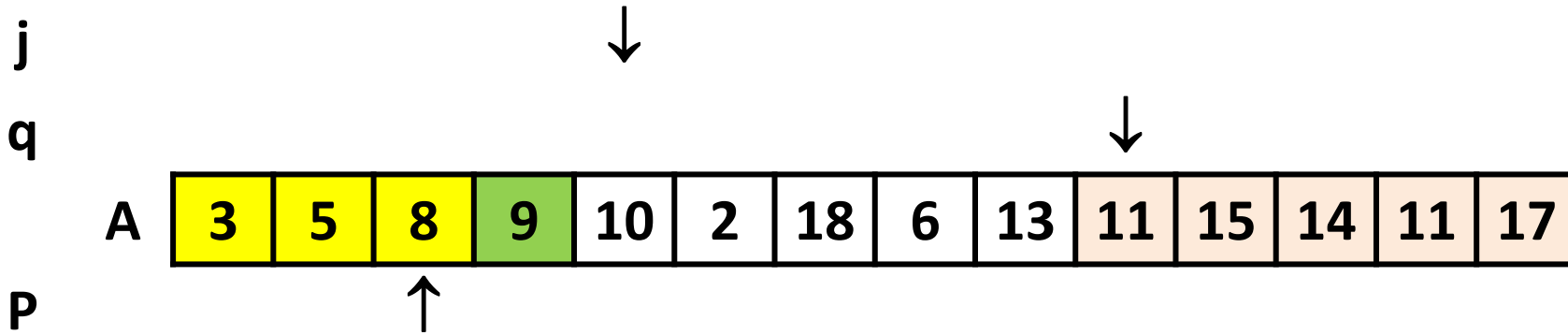


$A[j] = 9 = 9$ so $j++$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

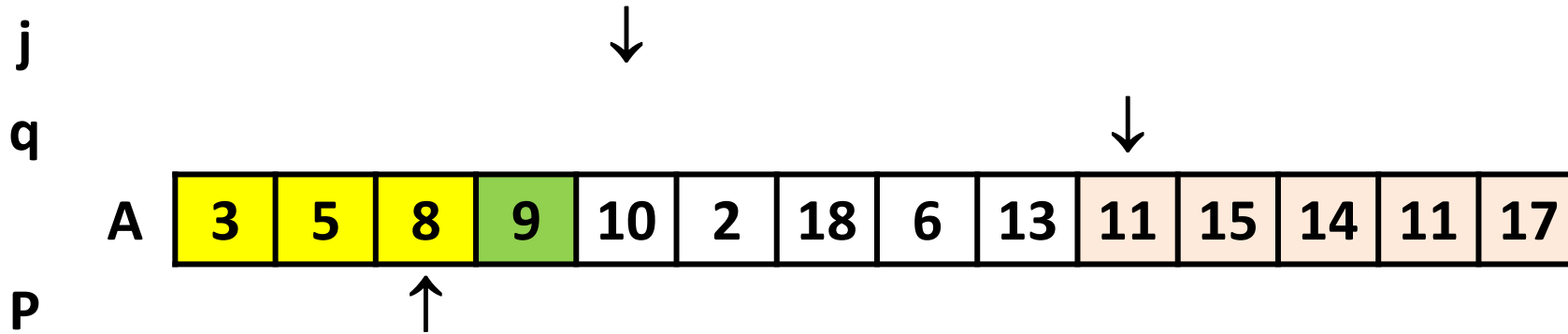


$A[j] = 9 = 9$ so $j++$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

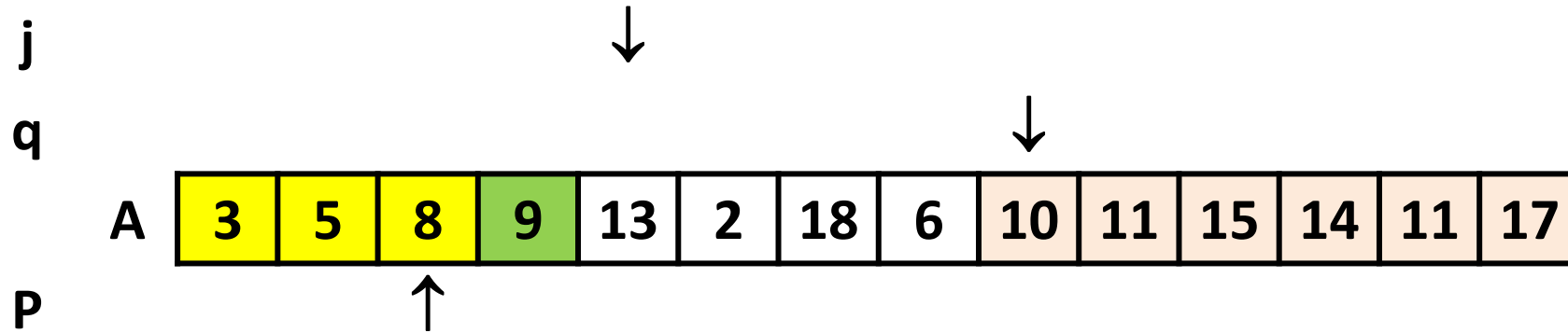


$A[j] = 10 > 9$ swap $A[j]$ and $A[--q]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

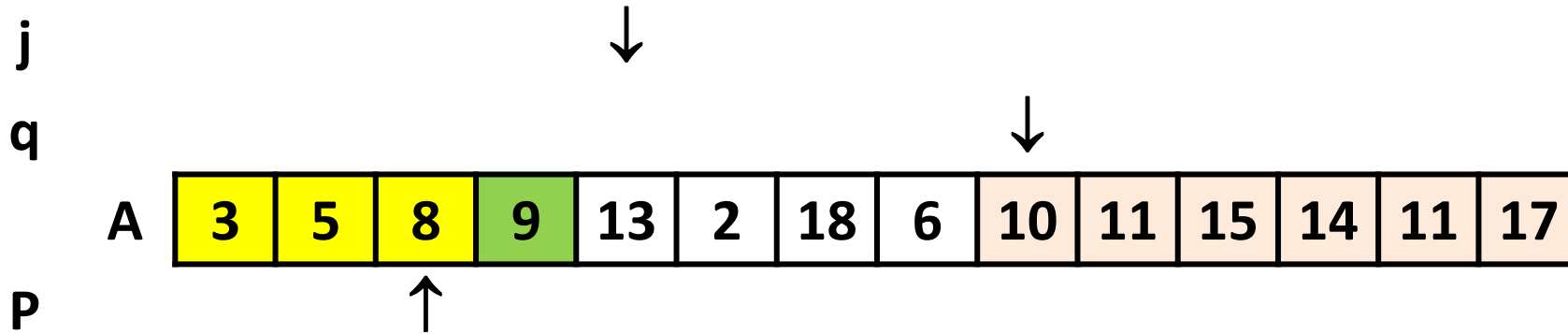


$A[j] = 10 > 9$ swap $A[j]$ and $A[--q]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

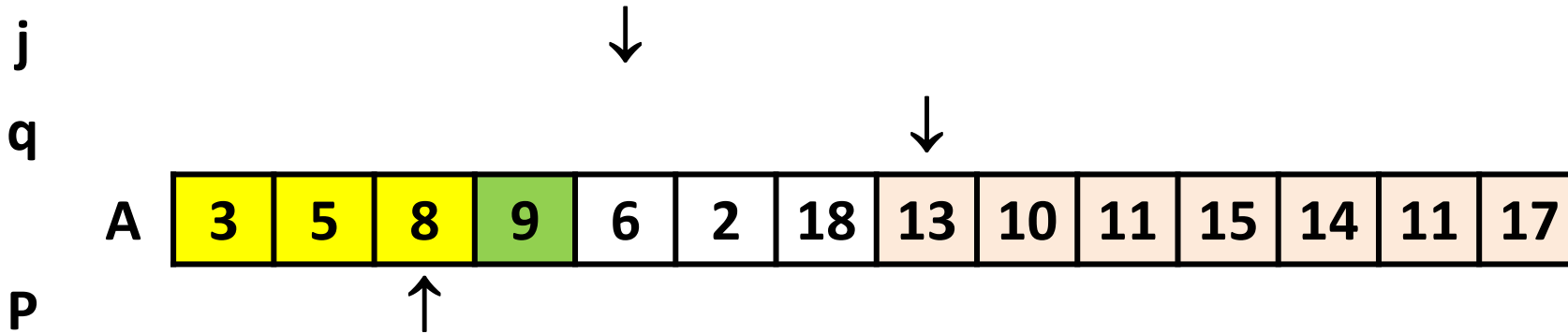


$A[j] = 13 > 9$ swap $A[j]$ and $A[--q]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

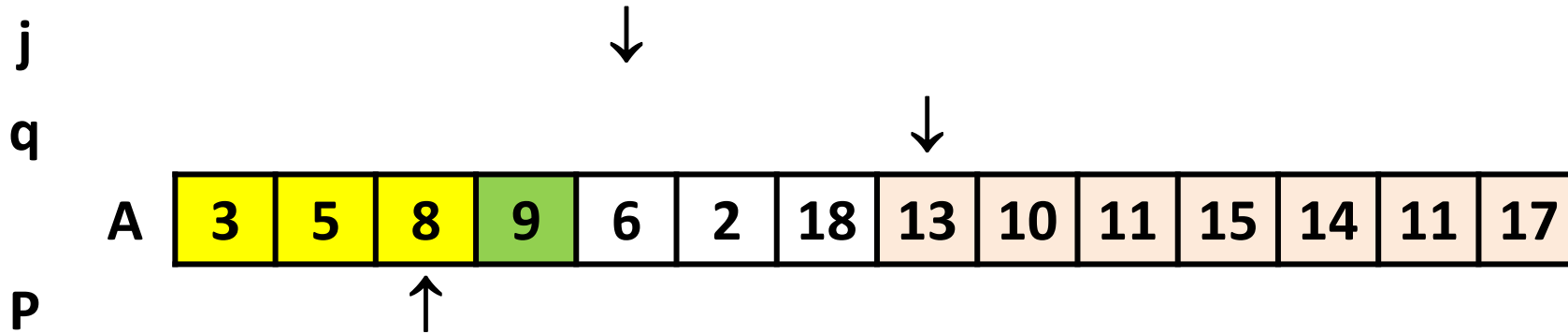


$A[j] = 13 > 9$ swap $A[j]$ and $A[q]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

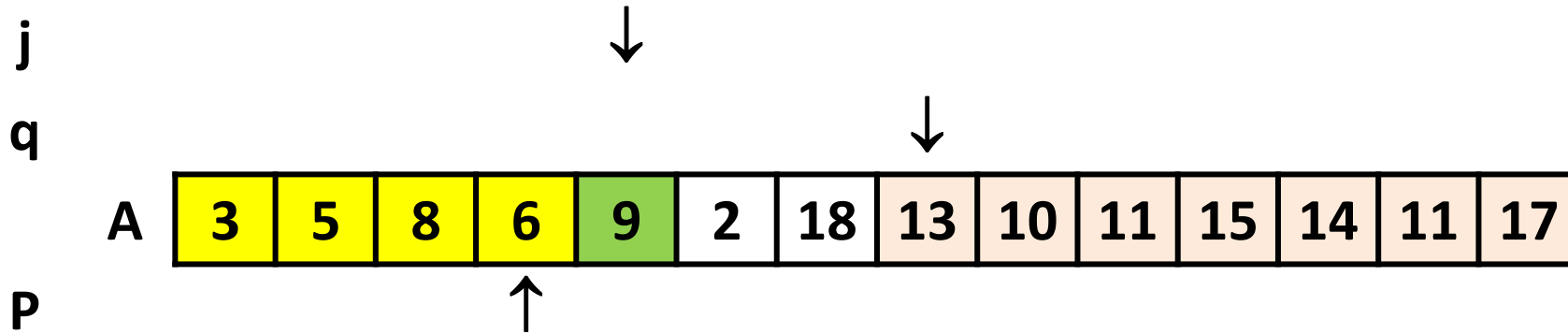


$A[j] = 6 < 9$ swap $A[j++]$ and $A[++p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

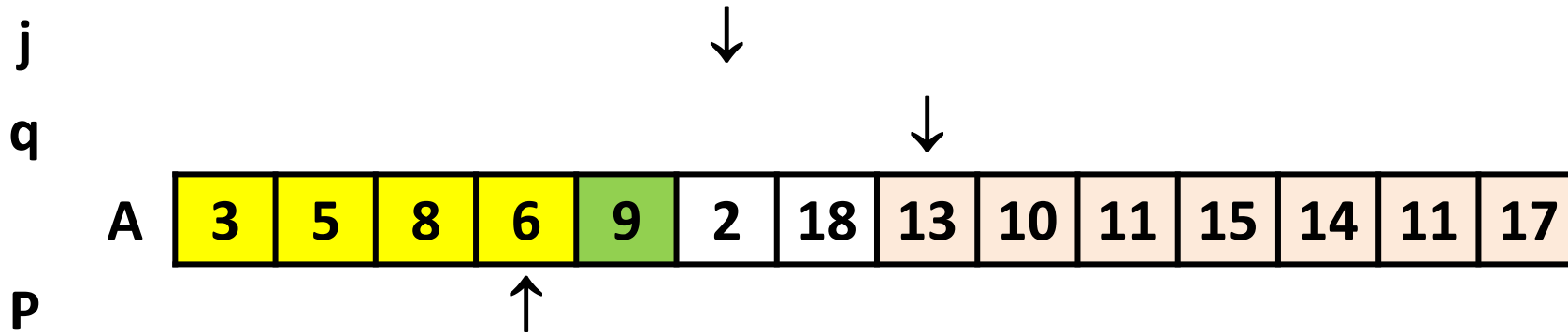


$A[j] = 6 < 9$ swap $A[j++]$ and $A[++p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

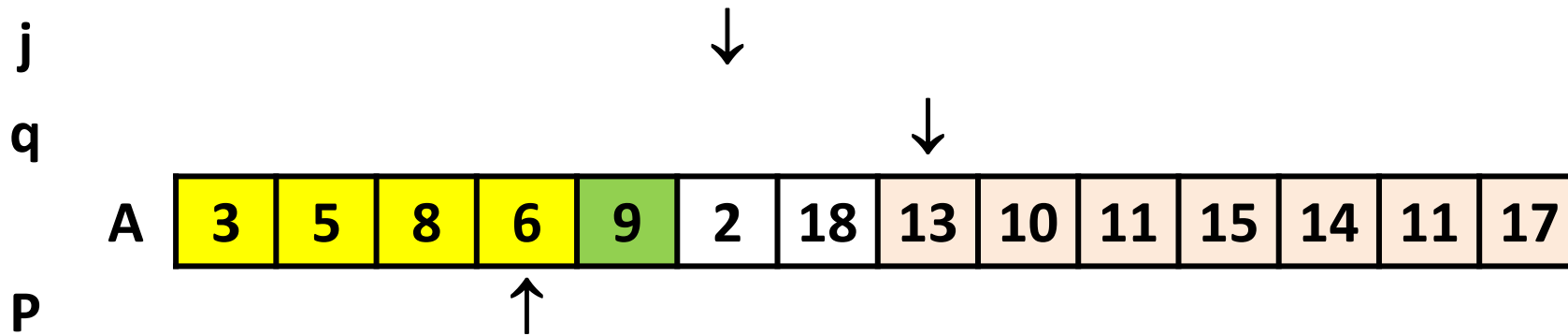


$A[j] = 6 < 9$ swap $A[j++]$ and $A[++p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

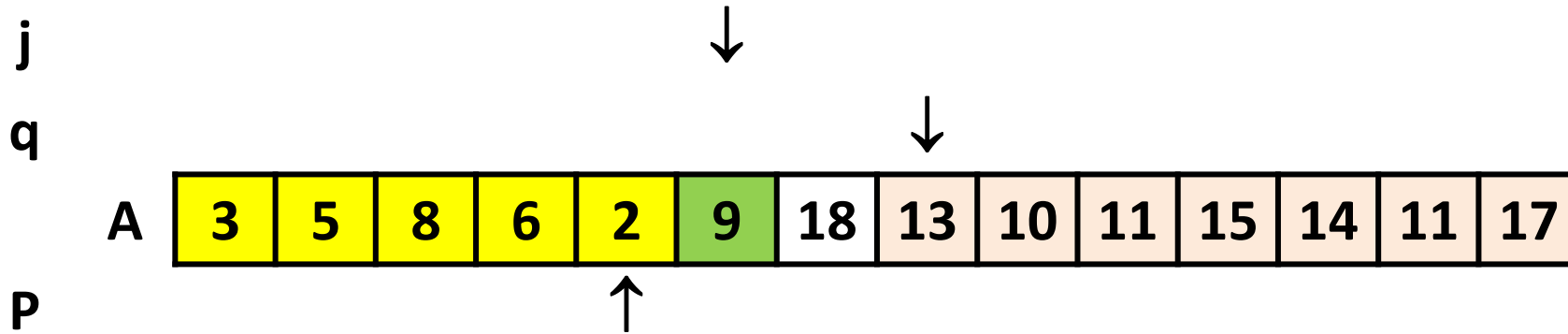


$A[j] = 2 < 9$ swap $A[j++]$ and $A[++p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

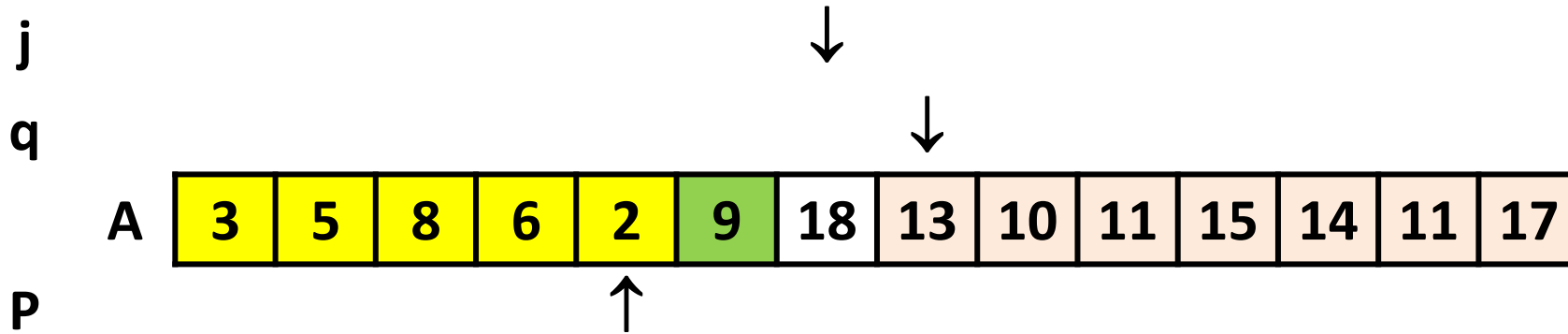


$A[j] = 2 < 9$ swap $A[j++]$ and $A[++p]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

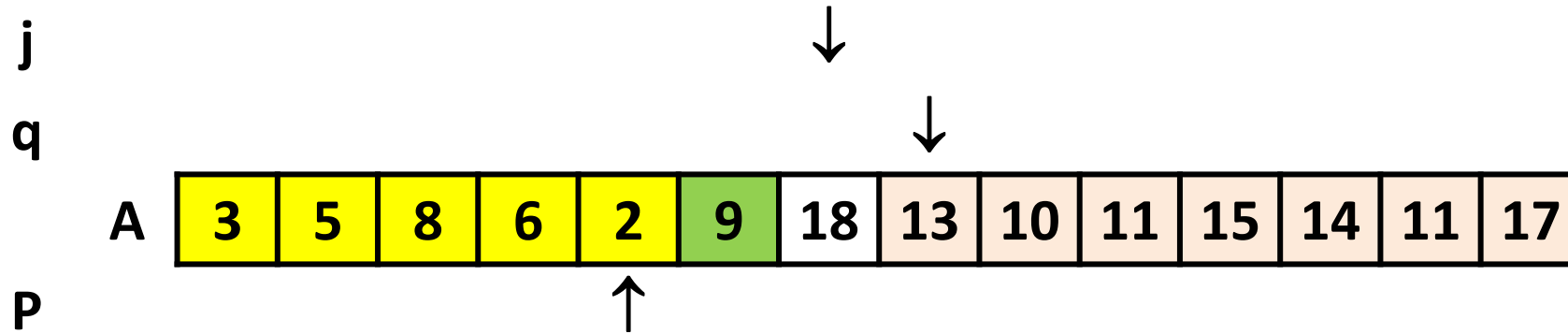


A[j] = 2 < 9 swap A[j++] and A[++p]

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

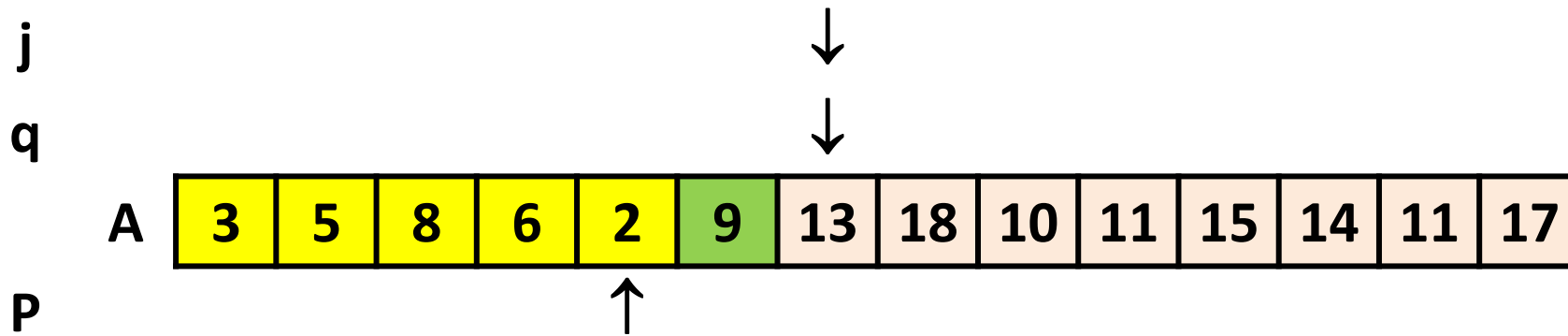


$A[j] = 18 > 9$ swap $A[j]$ and $A[--q]$

Quicksort

An efficient in-place partition function

Pivot is $x = 9$

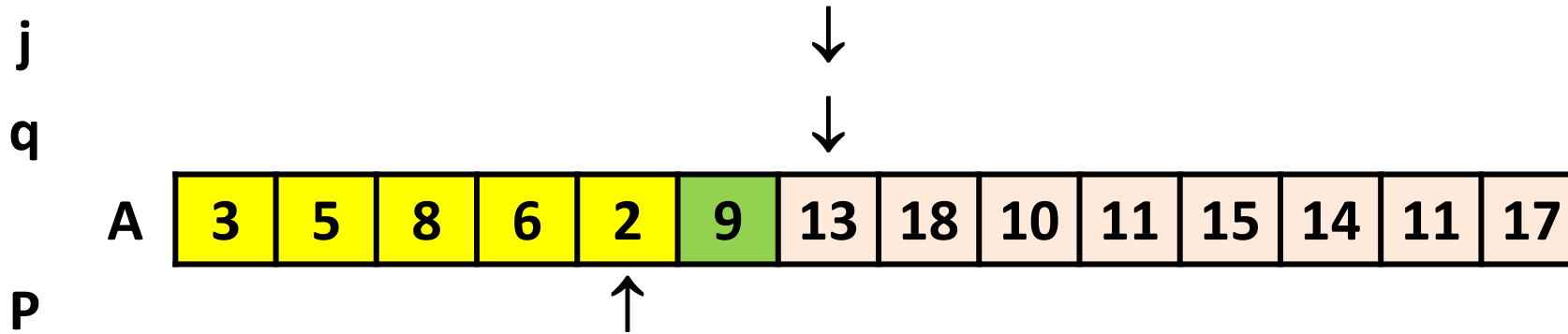


$A[j] = 18 > 9$ swap $A[j]$ and $A[--q]$

Quicksort

An efficient in-place partition function

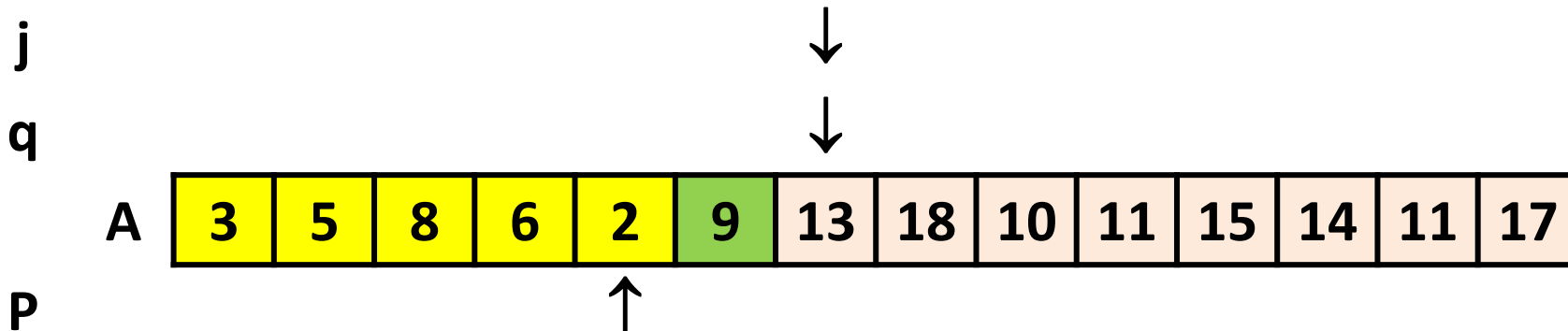
Pivot is $x = 9$



$j \geq q$ so stop

Quicksort

```
while (j < q) {  
    int comp = compare(a[j], x);  
    if (comp < 0) { // move to beginning of array  
        swap(a, j++, ++p);  
    } else if (comp > 0) {  
        swap(a, j, --q); // move to end of array  
    } else {  
        j++; // keep in the middle  
    }  
}
```



Quicksort

quicksort(A, i, n) overview

if $n \leq 1$ then return // base case does nothing

$p :=$ random element in $A[i..1+n]$

partition($A[i..i+n]$, p)

$k :=$ index of p in A // assert $i \leq k \leq i+n$

quicksort(A , i , $k-i$)

quicksort(A , $k+1$, $n-k$)

return

Quicksort

```
void quickSort(a) { quickSort(a, 0, a.length); }  
void quickSort(T[] a, int i, int n) {  
    if (n <= 1) return;  
    T x = a[i + rand.nextInt(n)];  
    int p = i-1, j = i, q = i+n;  
    // a[i..p]<x, a[p+1..q-1]?x, a[q..i+n-1]>x  
    while (j < q) { int comp = compare(a[j], x);  
        if (comp < 0) { // move to beginning of array  
            swap(a, j++, ++p);  
        } else if (comp > 0) {  
            swap(a, j, --q); // move to end of array  
        } else {  
            j++; // keep in the middle  
        }  
    }  
    // a[i..p]<x, a[p+1..q-1]=x, a[q..i+n-1]>x  
    quickSort(a, i, p-i+1, c);  
    quickSort(a, q, n-(q-i), c);  
}
```

Quicksort

Consider calling quicksort on the numbers
 $0, 1, \dots, n - 1$

Let's look at the quicksort recursion tree
(labelling nodes with the partition value)

Quicksort

Consider calling quicksort on the numbers
 $0, 1, \dots, n - 1$

Let's look at the quicksort recursion tree
(labelling nodes with the partition value)

Quicksort

Consider calling quicksort on the numbers
 $0, 1, \dots, n - 1$

Let's look at the quicksort recursion tree
(labelling nodes with the partition value)

The quicksort recursion tree is a random binary search tree!

Quicksort

Recall Lemma 7.1 (slightly rephrased):

In a random binary search tree of size n , the expected depth of $x \in \{0, \dots, n-1\}$ is

$$H_{x+1} + H_{n-x} - O(1)$$

Quicksort

Recall Lemma 7.1 (slightly rephrased):

In a random binary search tree of size n , the expected depth of $x \in \{0, \dots, n-1\}$ is

$$H_{x+1} + H_{n-x} - O(1)$$

Lemma 11.1: When quicksort is called to sort an array containing the integers $0, 1, \dots, n-1$, the expected number of times element x is compared to a pivot element is at most $H_{x+1} + H_{n-x}$.

Quicksort

Theorem 11.2: When quicksort sorts an array of n distinct elements, the expected number of comparisons is at most $2n \ln n + O(n)$.

Proof:

$$E = \sum_{i=0}^{n-1} (H_{i+1} + H_{n-i}) = 2 \sum_{i=1}^n H_i \leq 2 \sum_{i=1}^n H_n$$

$$E \leq 2n \ln(n) + 2n = 2n \ln(n) + O(n)$$

Comparison-Based Sorting

	Comparisons		In-place	Stable
Mergesort	$n \log(n)$	Worst-case	No	Yes
Quicksort	$1.38 n \log(n) + O(n)$ $O(n^2)$	Expected Worst-case	Yes	No
Heapsort	$2n \log(n) + O(n)$	Worst-case	Yes	No
Insertion Sort	$O(n^2)$ $O(n)$	Worst-case Best-case	Yes	Yes
Selection	$O(n^2)$	Worst-case	Yes	Yes

Mergesort

Theorem 11.1: Mergesort runs in $O(n \log n)$ time and performs at most $n \log n$ comparisons.

```
mergesort(A)
```

```
    if A.size = 1 then return A
```

```
    left := mergesort( left side of A)
```

```
    right := mergesort( right side of A)
```

```
    return merge(left, right)
```

Quicksort

Theorem 11.3: Quicksort runs in $O(n \log n)$ expected time and the expected number of comparisons is at most $2n \ln(n) + O(n)$.

```
quicksort(A, i, n):  
    if n <= 1 then return  
    k := partition(A[i..i+n], random pivot)  
    quicksort(A, i, k-i)  
    quicksort(A, k+1, n-k)  
    return
```

Heapsort

Theorem 11.4: The **heapsort** method runs in $O(n \log(n))$ time and performs at most $2n \log(n) + O(n)$ comparisons.

```
heapsort(A):  
    heapify(A)  
    for i from n-1 down to 2  
        swap A[i] and A[0]  
        trickleDown(0)  
reverse(A)
```


Lower Bound for Sorting

Theorem 11.5: For any deterministic comparison-based sorting algorithm A and any integer $n \geq 1$, there exists an input array a of length n such that $A(a)$ performs at least

$$\log(n!) = n \log(n) - O(n)$$

comparisons when sorting a .

Lower Bound for Sorting

What is a comparison-based sorting algorithm?

Input: array A

Operations: make m comparisons $A[i] < A[j]$

Output: permutation σ that sorts the array.

$$A[\sigma_0] < A[\sigma_1] < \dots < A[\sigma_{n-1}]$$

Lower Bound for Sorting

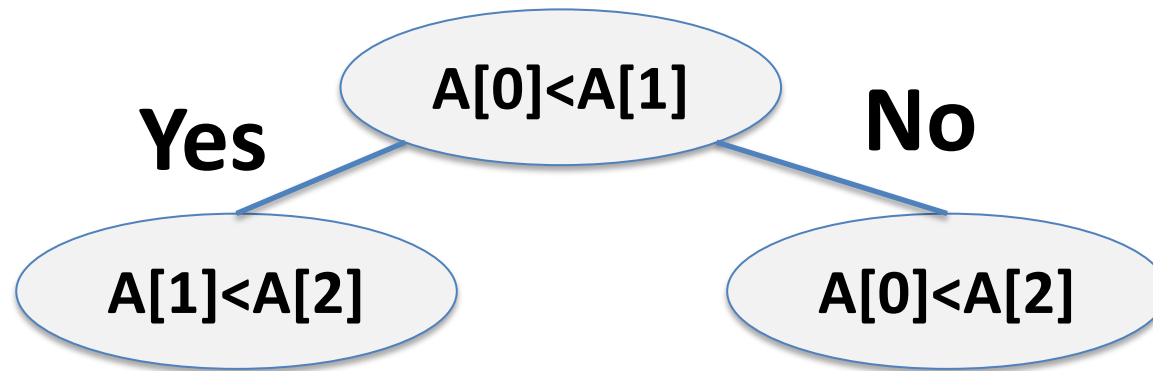
Suppose we have an array of 3 elements $\{a,b,c\}$.

Given $A = [b,c,a]$ the sorting algorithm must find the permutation $\sigma = 201$ so that

$$\begin{aligned} [A[\sigma_0], A[\sigma_1], A[\sigma_2]] &= [A[2], A[0], A[1]] \\ &= [a, b, c] \end{aligned}$$

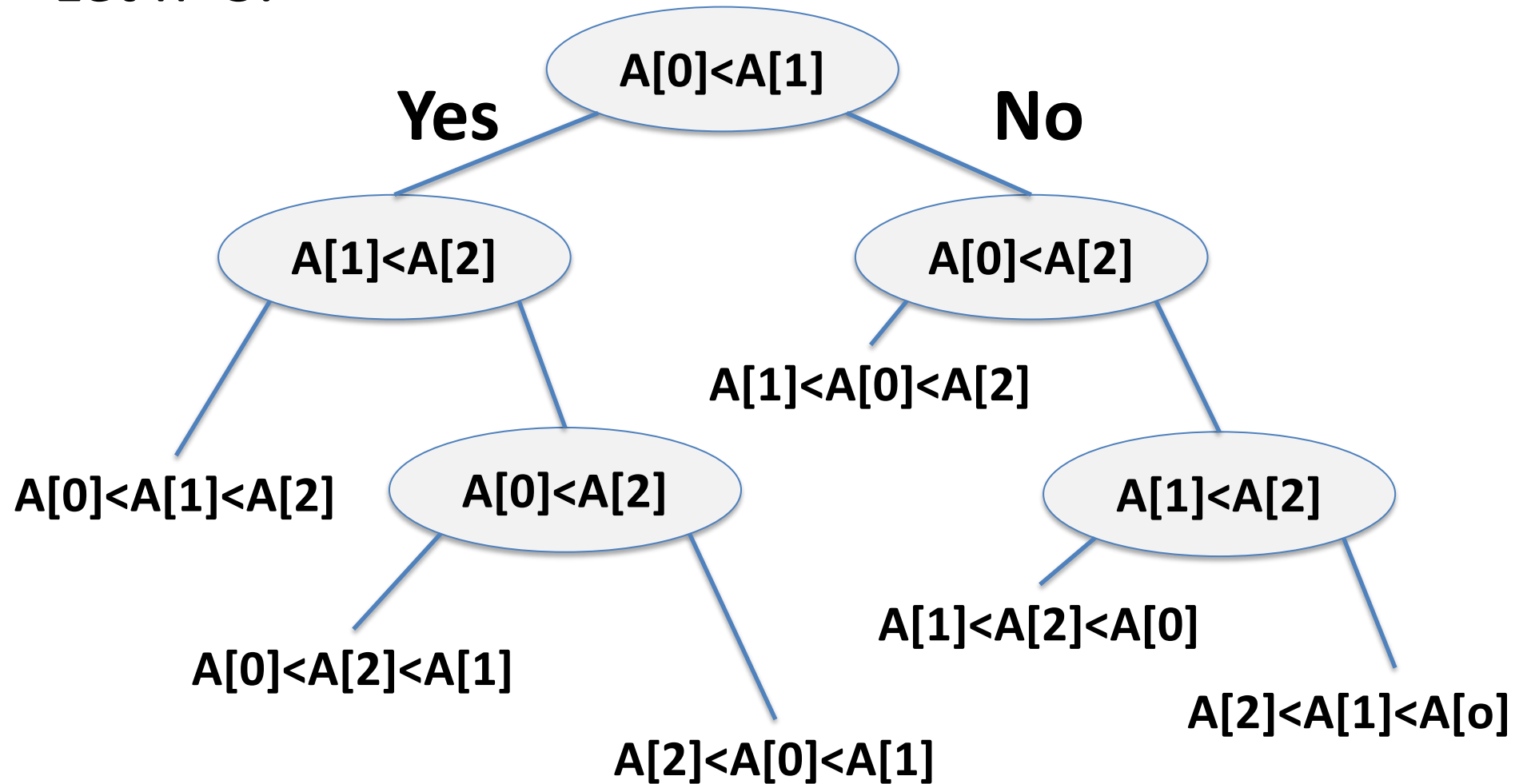
Lower Bound for Sorting

We model the sorting algorithm with a **decision tree**. Each node is a comparison of two array elements and each leaf is a permutation.



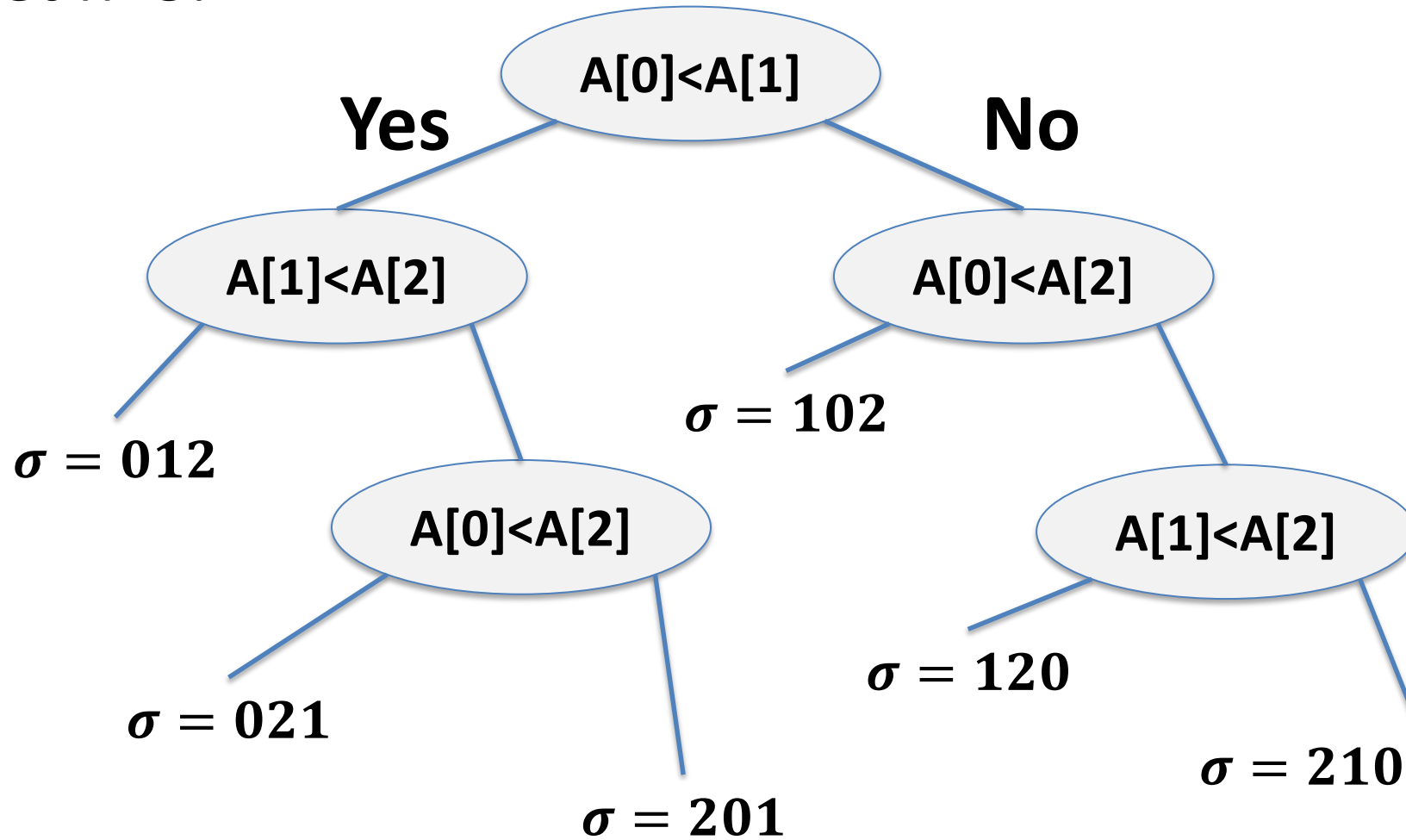
Lower Bound for Sorting

Let $n=3$.



Lower Bound for Sorting

Let $n=3$.



Lower Bound for Sorting

We **need** a binary tree with at least $n!$ leaves.
(At least one leaf for each permutation)

In the “best” case the longest search path (# of comparisons) is $\log(n!)$

Exercise: Show that

$$\log(n!) = n \log(n) - O(n) \in O(n \log n)$$

Timsort

The default sort algorithm in Python, Java, Swift, Rust and others.

Timsort is a hybrid stable sorting algorithm based on mergesort and insertion sort.

Timsort runs in $O(n \log(n))$ worst-case time, has $O(n)$ best-case time, and uses up to $O(n)$ extra memory.

Comparison-Based Sorting

	Comparisons		In-place	Stable
Mergesort	$n \log(n)$	Worst-case	No	Yes
Quicksort	$1.38 n \log(n) + O(n)$ $O(n^2)$	Expected Worst-case	Yes	No
Heapsort	$2n \log(n) + O(n)$	Worst-case	Yes	No
Timsort	$O(n \log n)$ $O(n)$	Worst-case Best-case	No	Yes
Insertion Sort	$O(n^2)$ $O(n)$	Worst-case Best-case	Yes	Yes
Selection	$O(n^2)$	Worst-case	Yes	Yes

Recursion

Do we really need (want) such small base case?

Consider mergesort.

Recursion

Better off to stop when lists have size 20 instead of 1 to cut down the number of recursive calls needed and use insertion sort.

Suppose $n = 2^{20} \approx 1,000,000$. Using a cut-off of 16 instead of 1 for recursion, how many calls to mergesort do we save?

Recursion

Suppose $n = 2^{20} \approx 1,000,000$. Using a cut-off of 16 instead of 1 for recursion, how many calls to mergesort do we save?